

Temario

1

Datos Multidimensionales

- ✓ Múltiples atributos
- ✓ Relaciones entre dimensiones

2

Relaciones Múltiples

- ✓ Matrices de dispersión
- ✓ Correlaciones

3

Representaciones Multidimensionales

- ✓ Coordenadas paralelas
- ✓ Perfiles y gráficos de radar

4

Reducción Dimensional

- ✓ Proyección y pérdida de información
- ✓ PCA & t-SNE
- ✓ Otras alternativas

Técnicas de Visualización — ¿Qué queremos observar?

► COMPARACIÓN

¿Cómo se comparan los valores entre categorías o grupos?

Barras · Puntos →

Facilitan comparar **magnitudes y posiciones**

► DISTRIBUCIÓN

¿Cómo se distribuyen los valores de un atributo?

Observaciones · Histograma · Boxplot · Violin plot →

Permiten observar **frecuencia, dispersión, forma y valores atípicos**

► RELACIONES

¿Cómo varían conjuntamente dos atributos?

Diagrama de dispersión · Correlación →

Permiten reconocer **dirección, fuerza, forma y casos particulares**

La elección de una representación depende de la tarea y de qué características de los datos necesitamos hacer visibles



De los Datos a la Representación

DATOS

¿Qué elementos y atributos queremos analizar?

TAREA

¿Qué queremos descubrir o comunicar?

MAPEO

¿Qué marcas y canales utilizamos?

REPRESENTACIÓN

¿Cómo organizamos visualmente los elementos?

Un Dataset para Explorar

Penguins

Palmer Penguins es un dataset de pingüinos observados en el archipiélago Palmer, Antártida.

344 observaciones · 7 atributos seleccionados

Cuantitativos

Longitud del pico · Profundidad del pico · Longitud de la aleta · Masa corporal

Categoricos

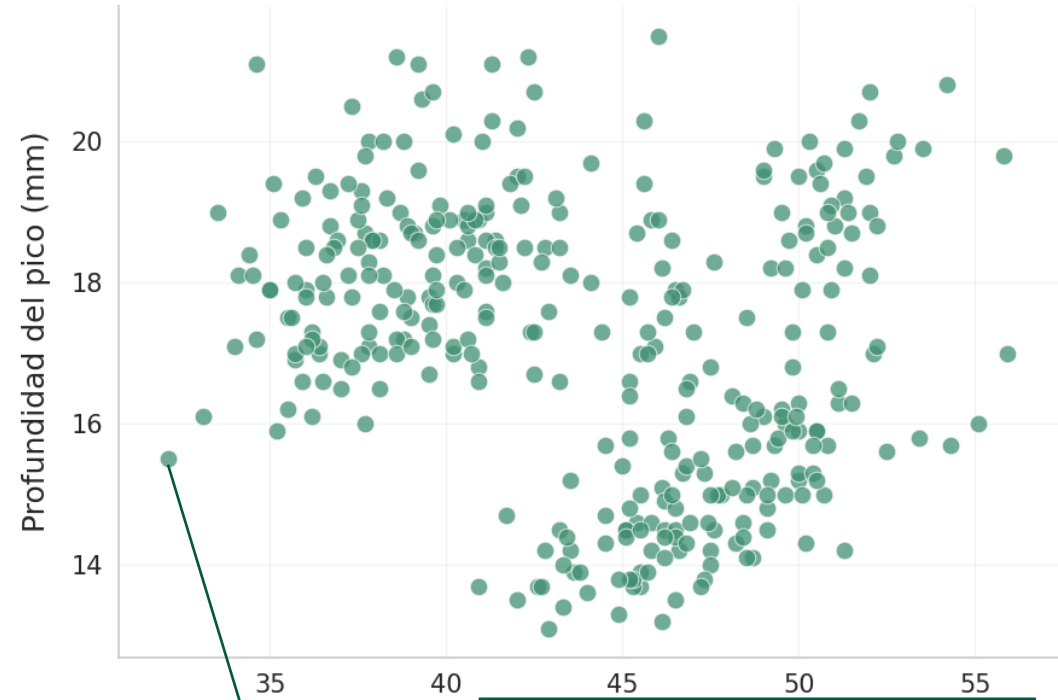
Especie · Isla · Sexo

species	Island (isla observación)	bill_length_mm (largo pico)	bill_depth_mm (profundidad pico)	flipper_length_mm (longitud aleta)	body_mass_g (masa corporal)	sex
Adelie	Torgersen	39.1	18.7	181	3750	Male
Adelie	Torgersen	39.5	17.4	186	3800	Female
Adelie	Torgersen	40.3	18.0	195	3250	Female
Adelie	Torgersen	36.7	19.3	193	3450	Female

Cada fila representa un pingüino · Cada columna representa un atributo

Visualizando dos Atributos

- ▶ Cada pingüino está caracterizado por múltiples atributos
- ▶ Podemos comenzar representando dos:
 - ✓ bill_length_mm → **Longitud del pico** → posición X
 - ✓ bill_depth_mm → **Profundidad del pico** → posición Y
- ▶ Scatterplot:
 - ✓ **Cada punto representa un pingüino**
 - ✓ Su posición está determinada por dos de sus atributos
- ▶ Pero también conocemos:
 - ✓ flipper_length_mm · body_mass_g · species · island · sex
 - ✓ Estos atributos **no están representados**



species	Adelie
island	Dream
bill_length_mm	32.1
bill_depth_mm	15.5
flipper_length_mm	188.0
body_mass_g	3050.0
sex	Female

¿Hay algún patrón que se pueda descubrir en la representación?



Visualizando Tres Atributos

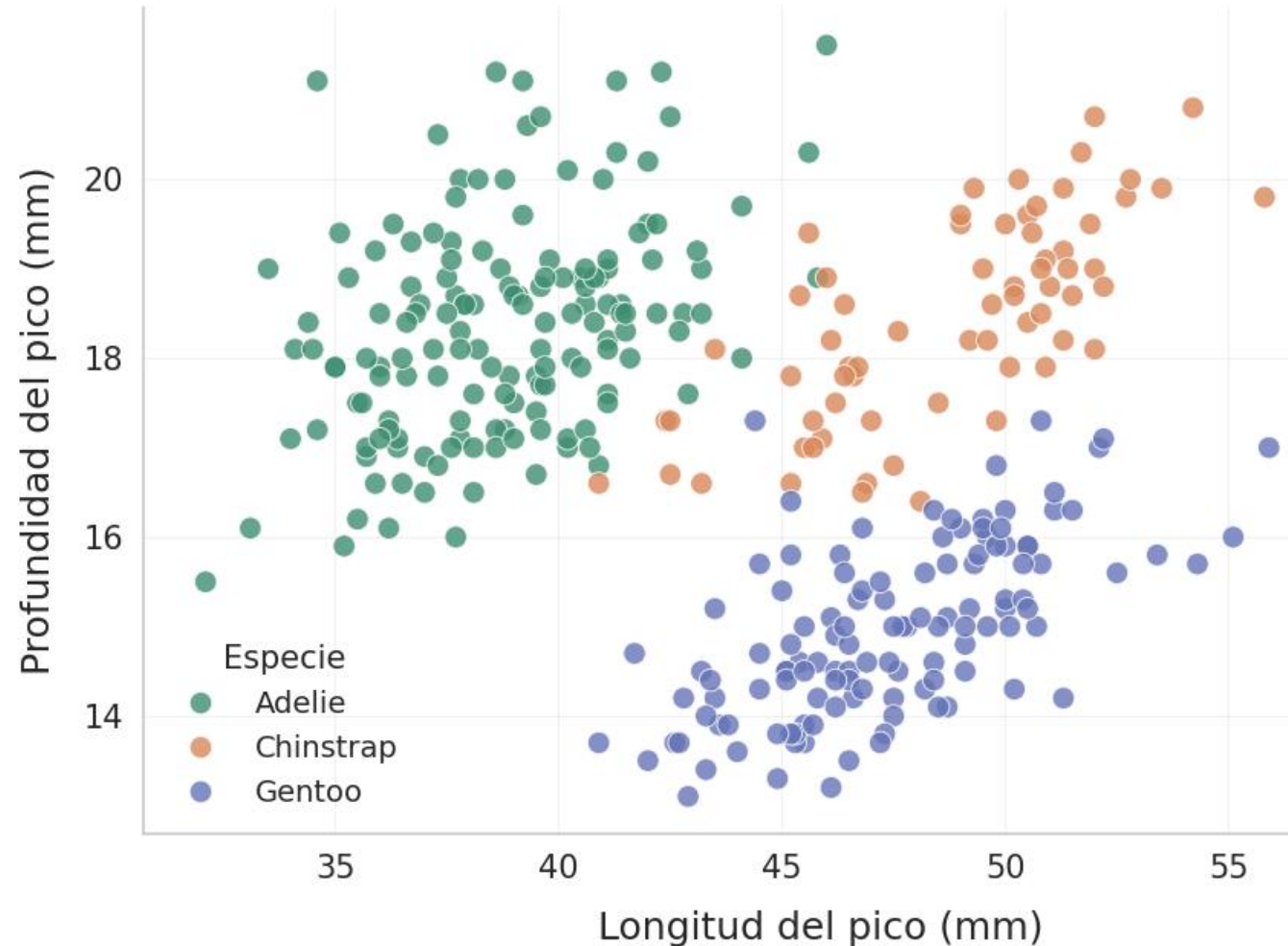
- Podemos representar atributos adicionales utilizando **otros canales visuales**:

- ✓ bill_length_mm → **Long. del pico** → X
- ✓ bill_depth_mm → **Prof. del pico** → Y
- ✓ species → **Especie** → color

- En el scatterplot:

- ✓ **Cada punto representa un pingüino**
- ✓ Ahora cada punto representa **tres atributos** del pingüino

¿Qué patrón que antes sólo intuíamos ahora se hace visible?



Visualizando Cuatro Atributos

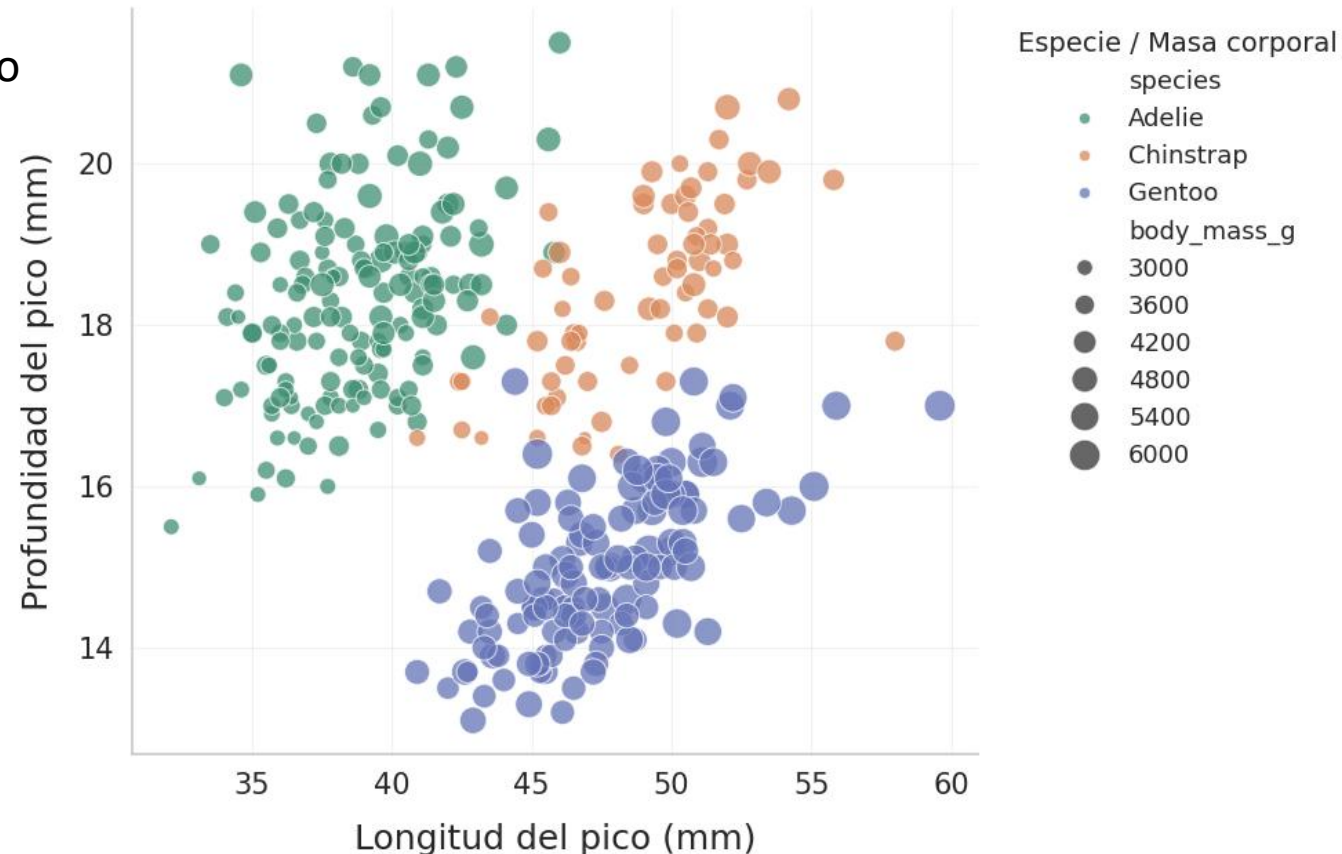
► Podemos representar atributos adicionales utilizando **otros canales visuales**:

- ✓ bill_length_mm → **Long. del pico** → X
- ✓ bill_depth_mm → **Prof. del pico** → Y
- ✓ species → **Especie** → color
- ✓ body_mass_g → **Masa corporal** → tamaño

► En el scatterplot:

- ✓ **Cada punto representa un pingüino**
- ✓ Ahora cada punto representa **cuatro atributos** de un pingüino

¿Los pingüinos de mayor masa corporal se concentran en alguna región del gráfico?



Efectividad del Mapeo Visual

► Mapeo **actual**:

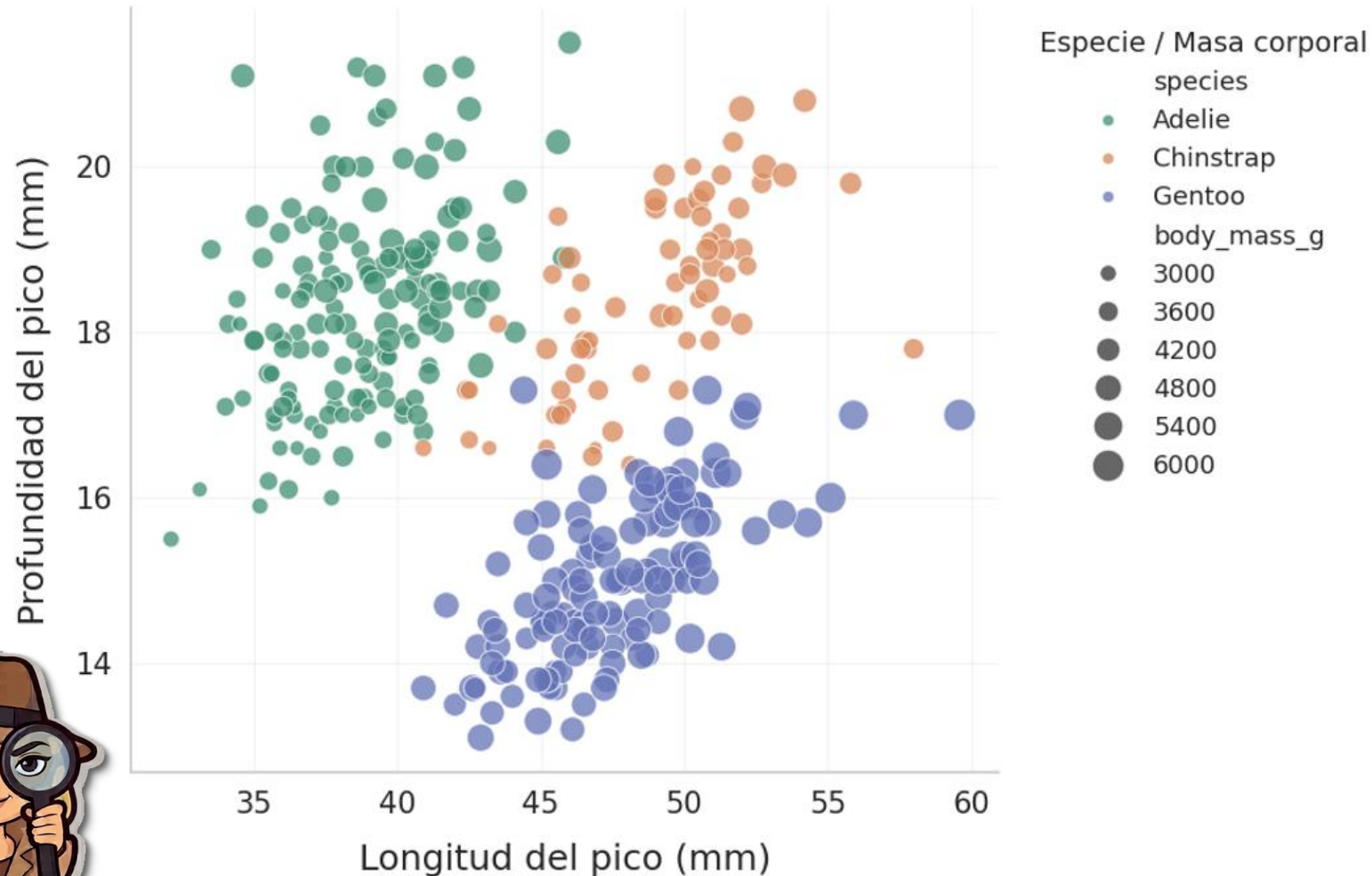
- ✓ Especie → **color**
- ✓ Masa corporal → **tamaño**

► ¿Qué tan expresivo y efectivo sería invertir estos mapeos?

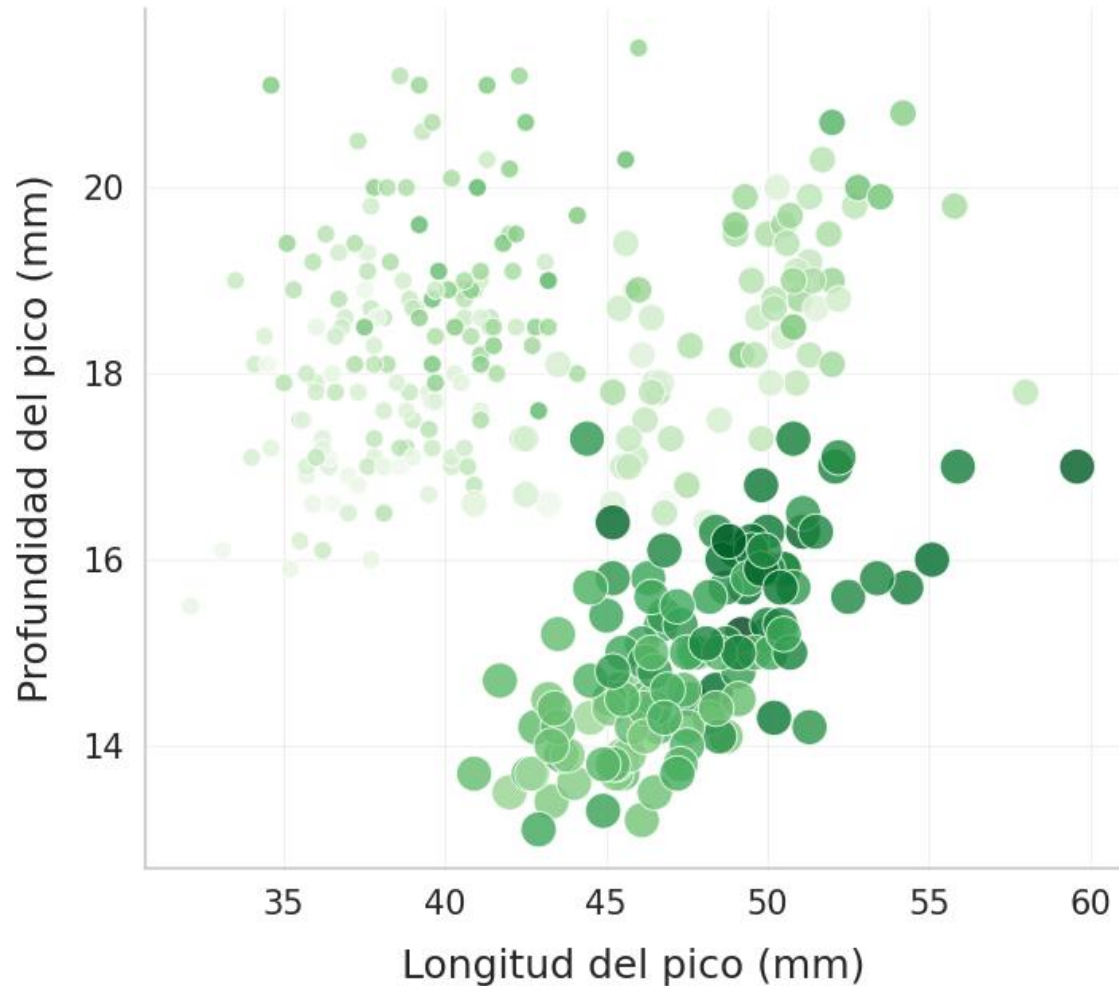
► Mapeo **Invertido**:

- ✓ Especie → **tamaño**
- ✓ Masa corporal → **color**

Mapeamos la especie con puntos de 3 tamaños y la masa corporal con una escala de colores



Efectividad del Mapeo Visual



- ▶ **Expresividad:** ¿El canal visual representa correctamente el tipo de dato?
 - ✓ Especie → **tamaño:** es problemático porque introduce una percepción de orden que las especies no tienen
 - ✓ Masa corporal → **color:** es correcto
- ▶ **Efectividad:** ¿El canal visual permite percibir y comparar los valores con facilidad?
 - ✓ Especie → **tamaño:** no es expresivo
 - ✓ Masa corporal → **color:** es posible pero dificulta estimar y comparar magnitudes

No alcanza con representar un atributo: el mapeo debe conservar sus propiedades y facilitar su interpretación

¿Podemos Seguir Agregando Dimensiones?

► Mapeo actual:

- ✓ bill_length_mm → **Long. del pico** → X
- ✓ bill_depth_mm → **Prof. del pico** → Y
- ✓ species → **Especie** → color
- ✓ body_mass_g → **Masa corporal** → tamaño

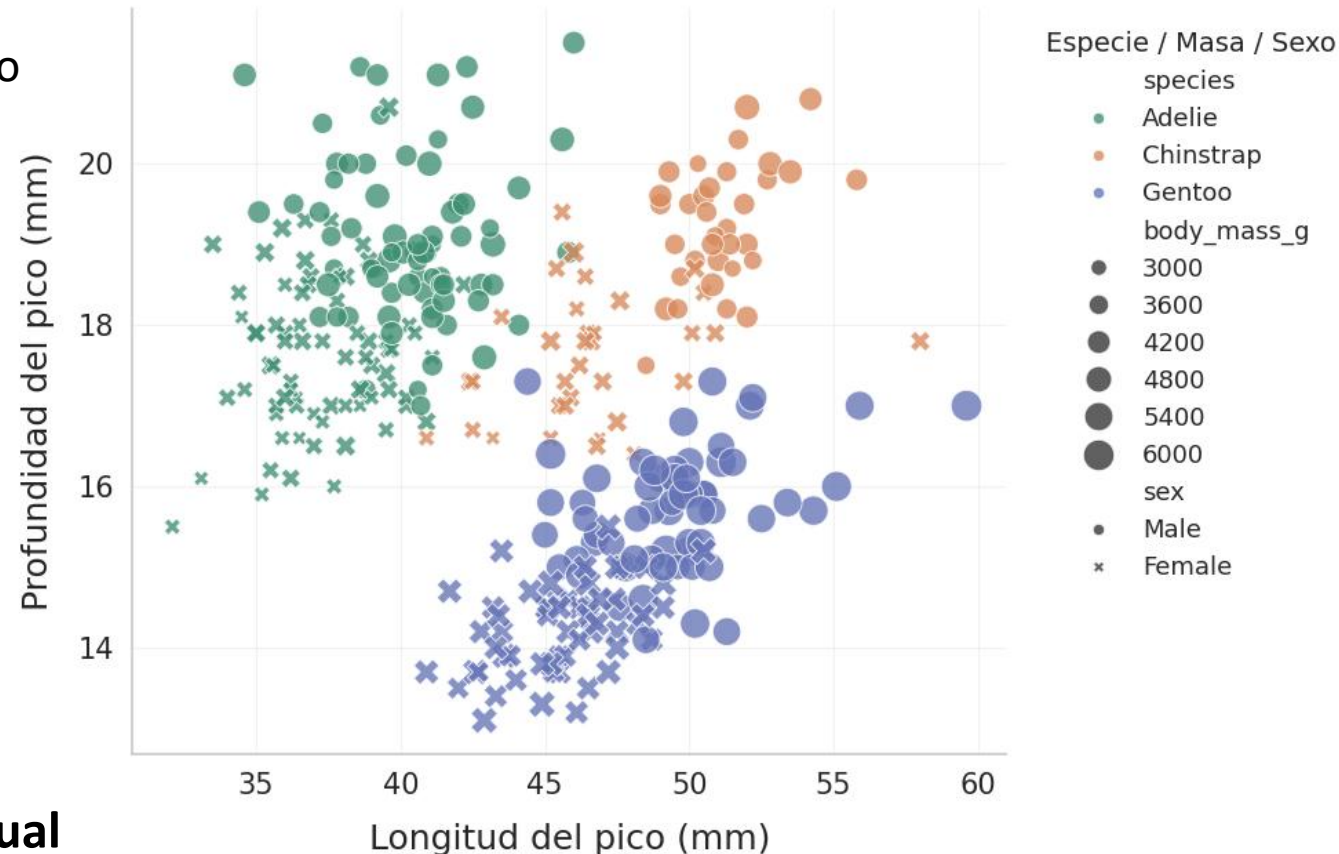
► Podemos agregar:

- ✓ sex → **Sexo** → forma

► ¿Resulta más fácil comprender los datos?

- ¿Podemos identificar especies?
- ¿Podemos comparar masas?
- ¿Podemos distinguir el sexo?
- ¿Podemos analizar simultáneamente todas estas relaciones?

► Cada nuevo canal aumenta la carga perceptual y dificulta algunas tareas



Muchas Relaciones, de a Dos

► Se puede construir un gráfico para observar **cada par de atributos cuantitativos**

► **Matriz de diagramas de dispersión**

Scatterplot Matrix / Pair Plot

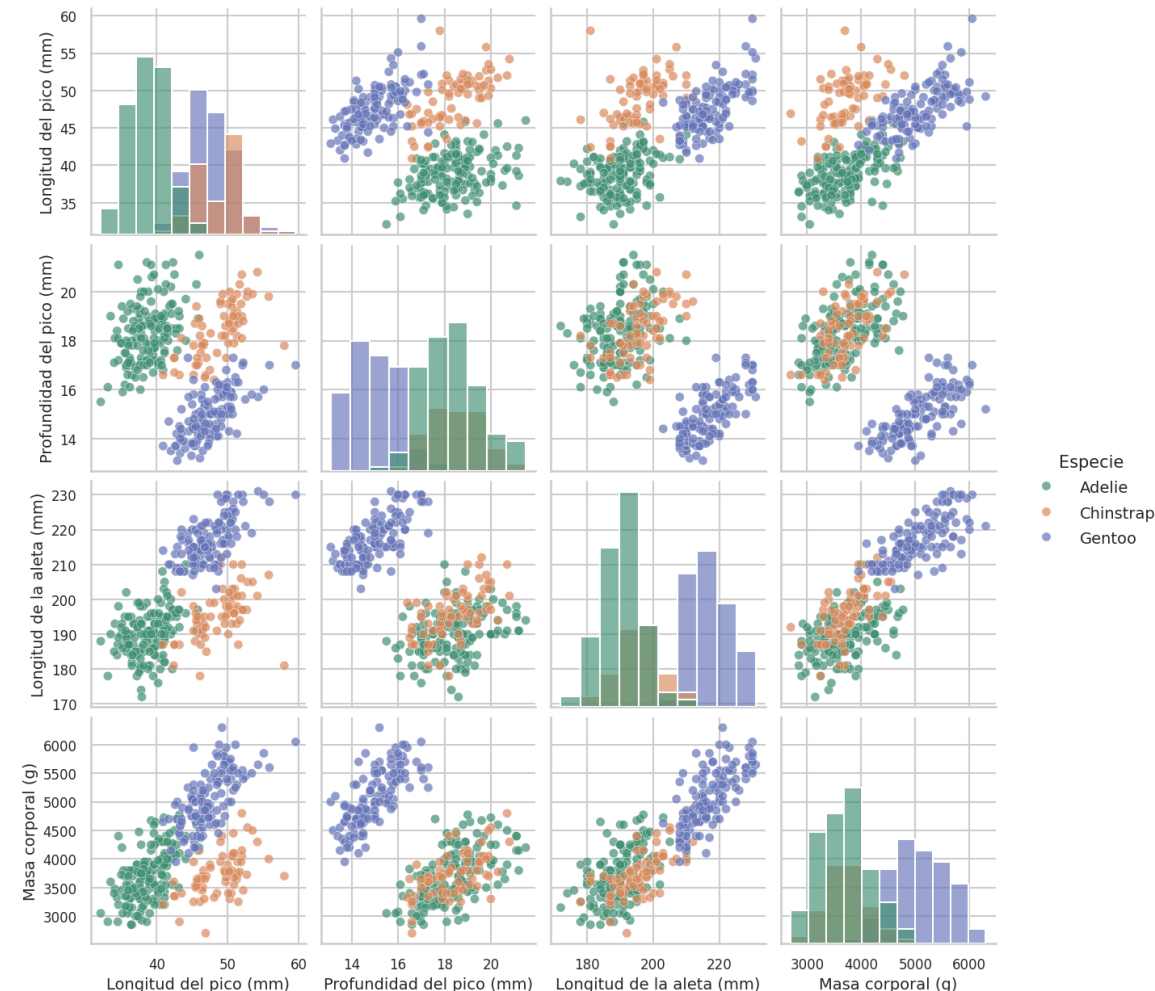
► Cada celda → relación entre dos atributos
Diagonal → distribución de cada atributo
Color → especie

► La matriz es **simétrica**: la relación A–B aparece como B–A (información repetida)

► La cantidad de gráficos crece rápidamente:

- 4 atributos → 6 pares de relaciones
- 10 atributos → **45 pares** distintos

► La matriz explora muchas relaciones, pero su complejidad crece rápidamente con el número de dimensiones



Resumiendo Relaciones

- ▶ La matriz de dispersión muestra cada relación en detalle
- ▶ Podemos **resumir cada par de atributos** mediante su correlación

	Longitud del pico	Profundidad del pico	Longitud de la aleta	Masa corporal
Longitud del pico	1,00	-0,24	0,66	0,60
Profundidad del pico	-0,24	1,00	-0,58	-0,47
Longitud de la aleta	0,66	-0,58	1,00	0,87
Masa corporal	0,60	-0,47	0,87	1,00

Cada celda → correlación entre dos atributos

- ✓ $r \approx +1$ → relación lineal positiva
- ✓ $r \approx 0$ → poca relación lineal
- ✓ $r \approx -1$ → relación lineal negativa

¿Qué pares de atributos presentan las relaciones más fuertes?



Mapa de Calor de Correlaciones

► Una matriz de correlación puede representarse mediante un **mapa de calor** (*heatmap*)

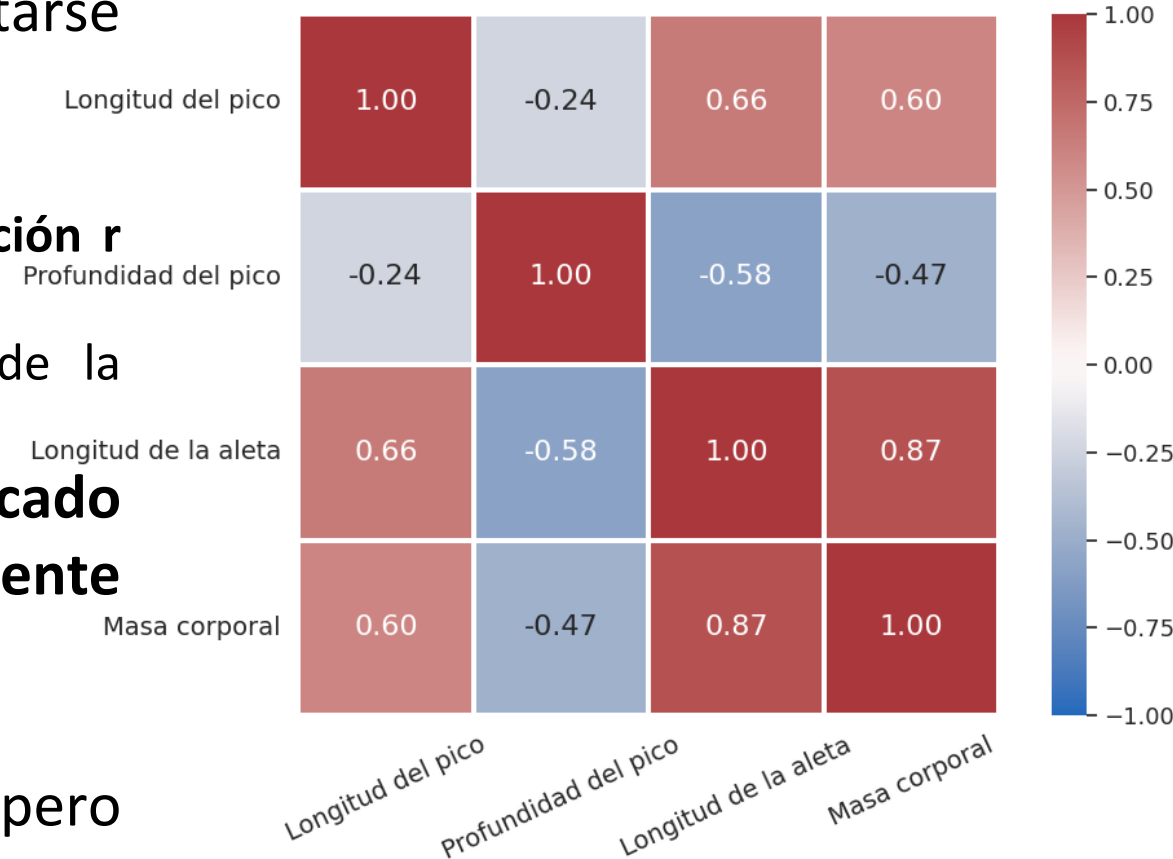
► Mapeo:

- ✓ La posición X e Y (celda) representa la **correlación r entre dos atributos**
- ✓ El **color** muestra la dirección e intensidad de la correlación

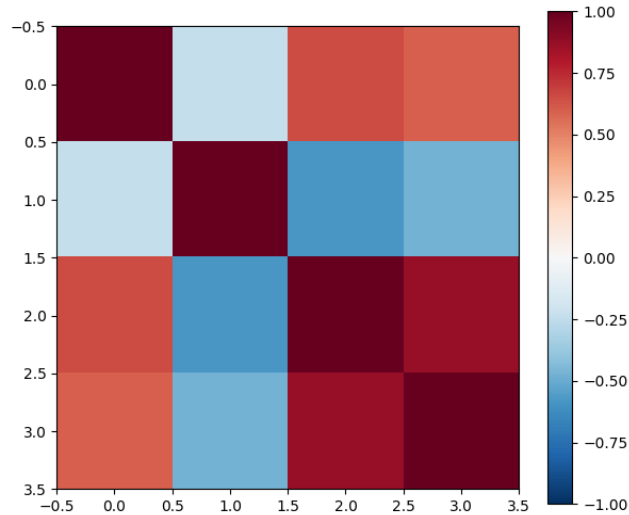
► r varia entre **-1 y +1** y **0** tiene un **significado particular**, se debe usar una **escala divergente centrada en 0**

► ⚠ El color facilita detectar patrones, pero dificulta comparar valores cercanos, por esto ubicamos los valores en la celda

► Heatmap = matriz + mapeo de valores a color

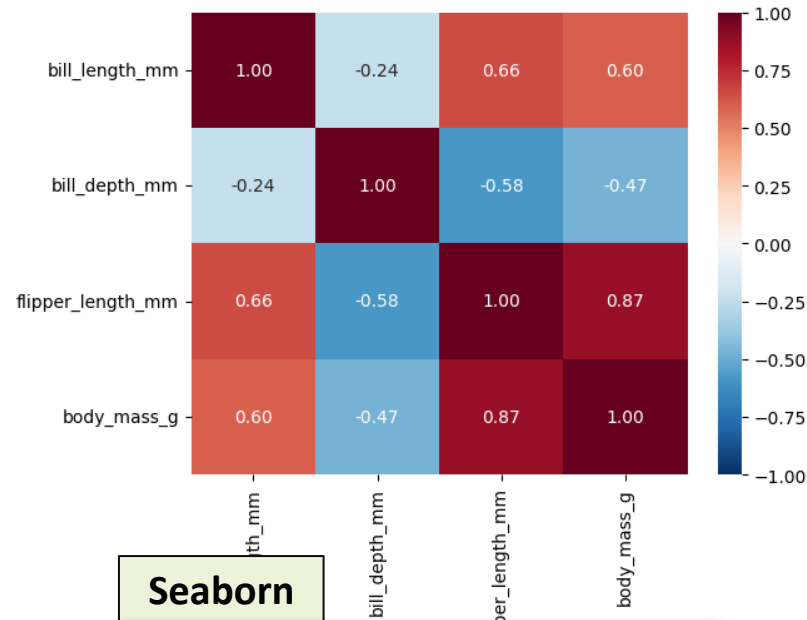


Mapa de Calor - Herramientas



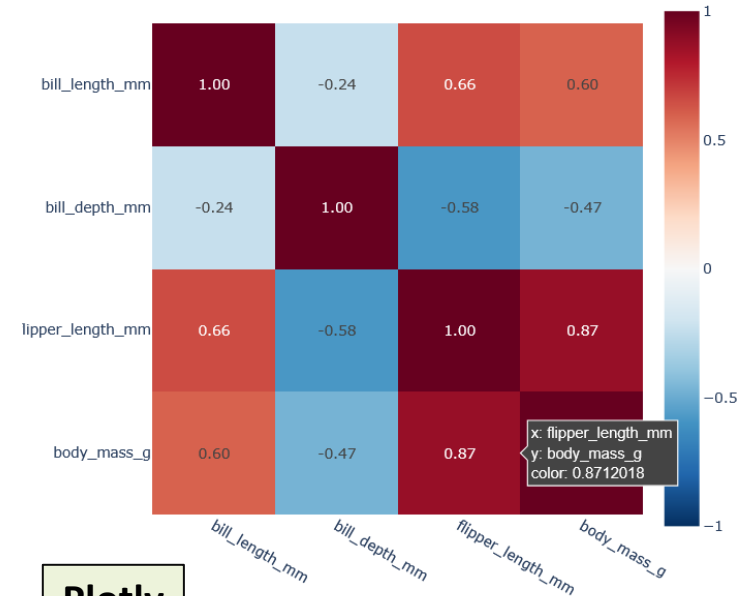
Matplotlib

```
im = ax.imshow(  
    corr,  
    cmap="RdBu_r",  
    vmin=-1,  
    vmax=1  
)  
fig.colorbar(im, ax=ax)
```



Seaborn

```
sns.heatmap(  
    corr,  
    annot=True,  
    fmt=".2f",  
    cmap="RdBu_r",  
    vmin=-1,  
    vmax=1,  
    center=0  
)
```



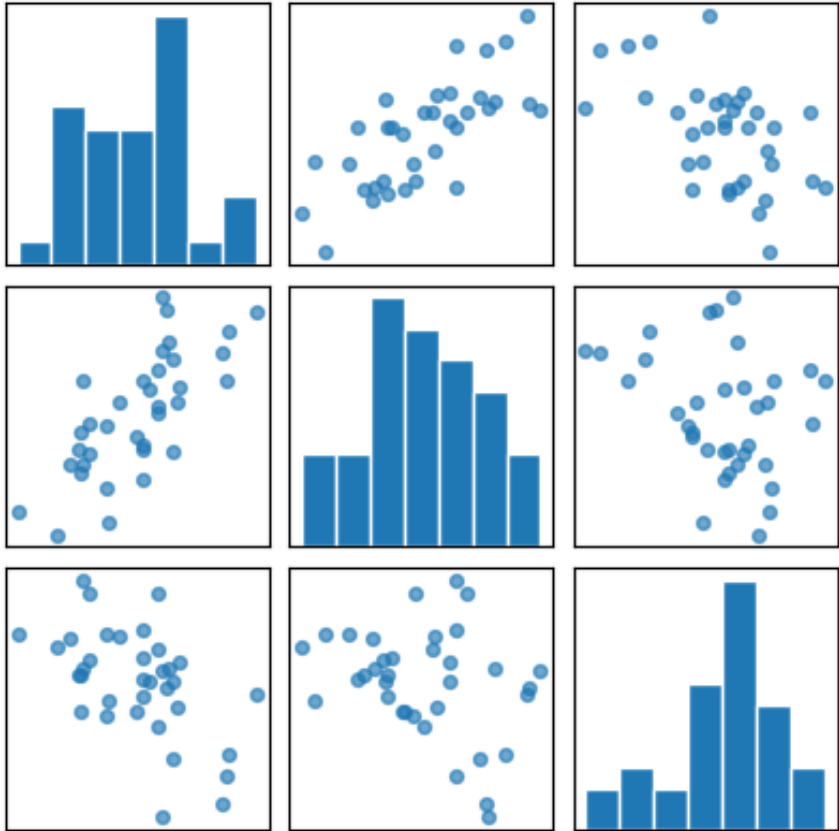
Plotly

```
fig = px.imshow(  
    corr,  
    text_auto=".2f",  
    color_continuous_scale="RdBu_r",  
    zmin=-1,  
    zmax=1,  
    color_continuous_midpoint=0)
```

Observando Varias Dimensiones a la Vez

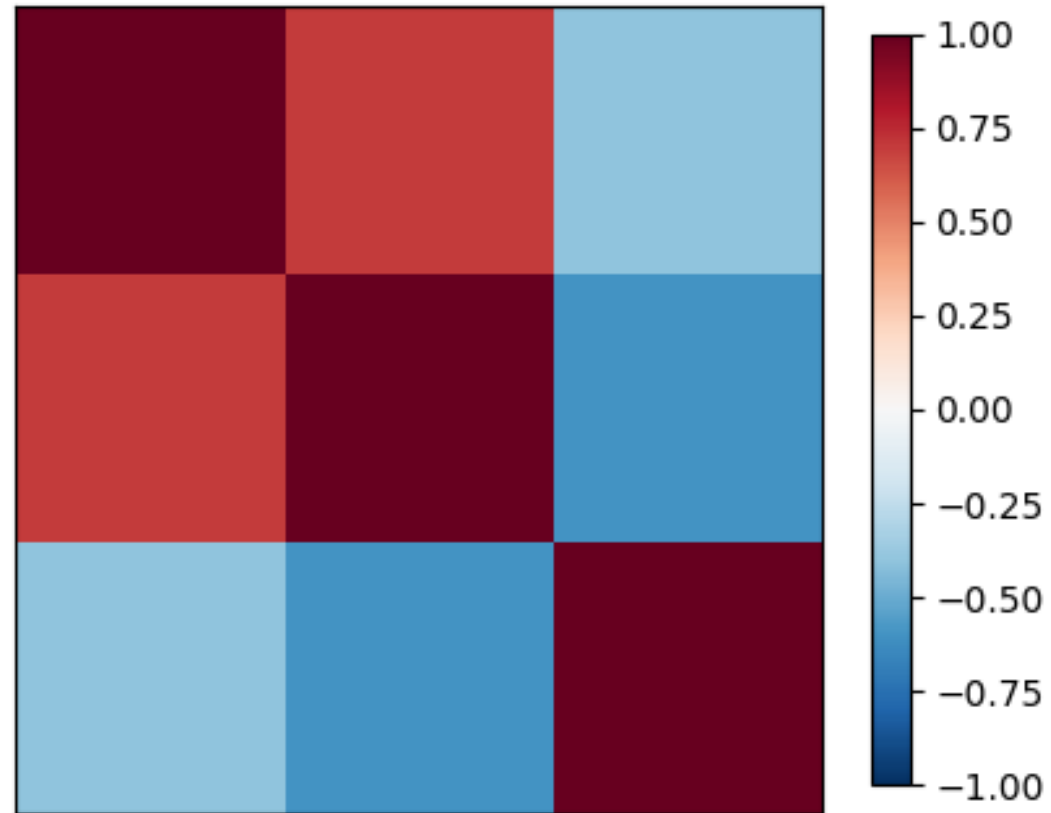
Scatterplot Matrix

Muchas relaciones → analizadas **de a pares**



Heatmap de correlación

Muchas relaciones → **resumidas** en un valor por par

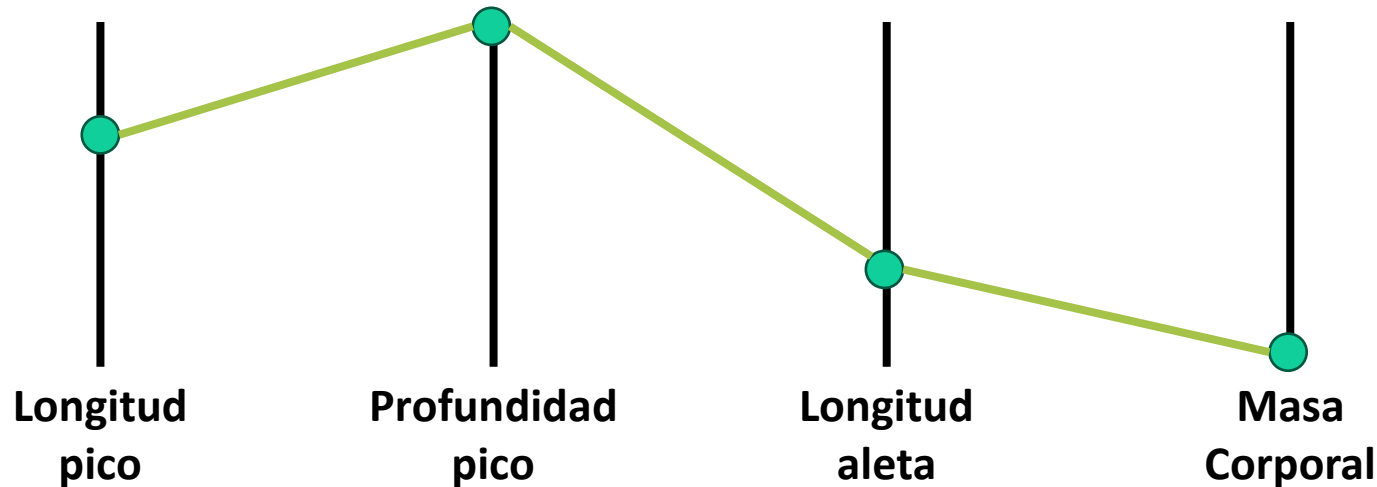


► ¿Podemos representar varios atributos de cada pingüino simultáneamente?

Una Observación, Muchos Atributos

- Cada fila contiene varios atributos de una misma observación
- Llevamos cada atributo a una escala común

Longitud pico	Profundidad pico	Longitud aleta	Masa corporal
39,1 mm	18,7 mm	181 mm	3750 g



¿Qué ocurriría si representamos de esta forma todos los pingüinos?

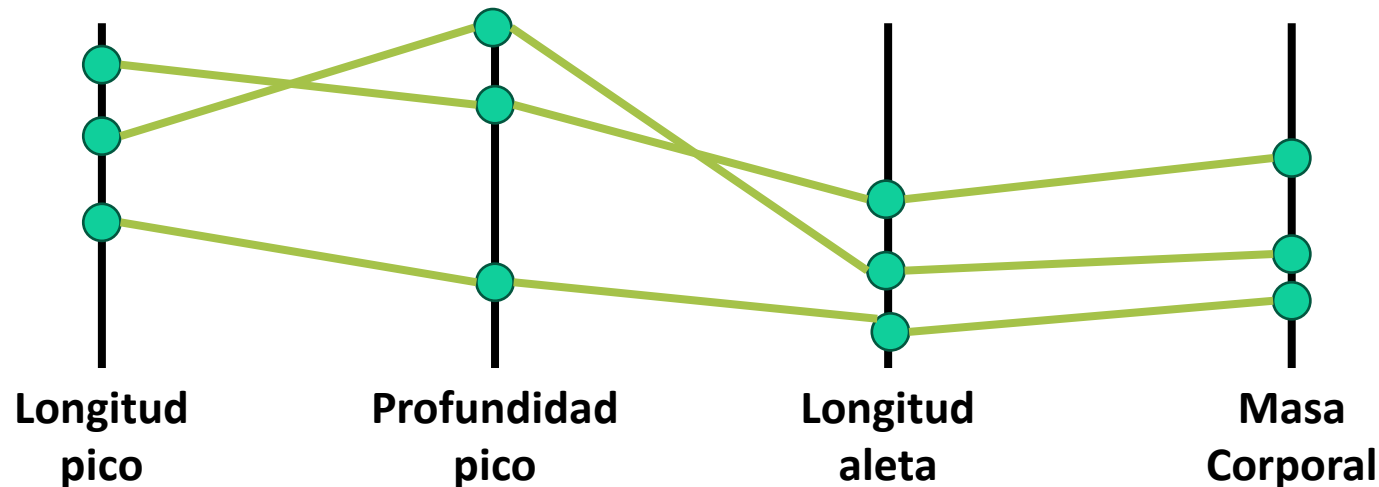


- Conectando los valores, obtenemos el perfil de la observación en varias dimensiones

Coordenadas Paralelas

- Cada atributo se representa mediante un eje paralelo y cada observación mediante una línea que conecta sus valores:

- ✓ Eje → atributo
Posición sobre el eje → valor
Línea → observación



Comparar atributos con diferentes unidades y rangos, requieren que sus valores estén en una **escala común**:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

con:

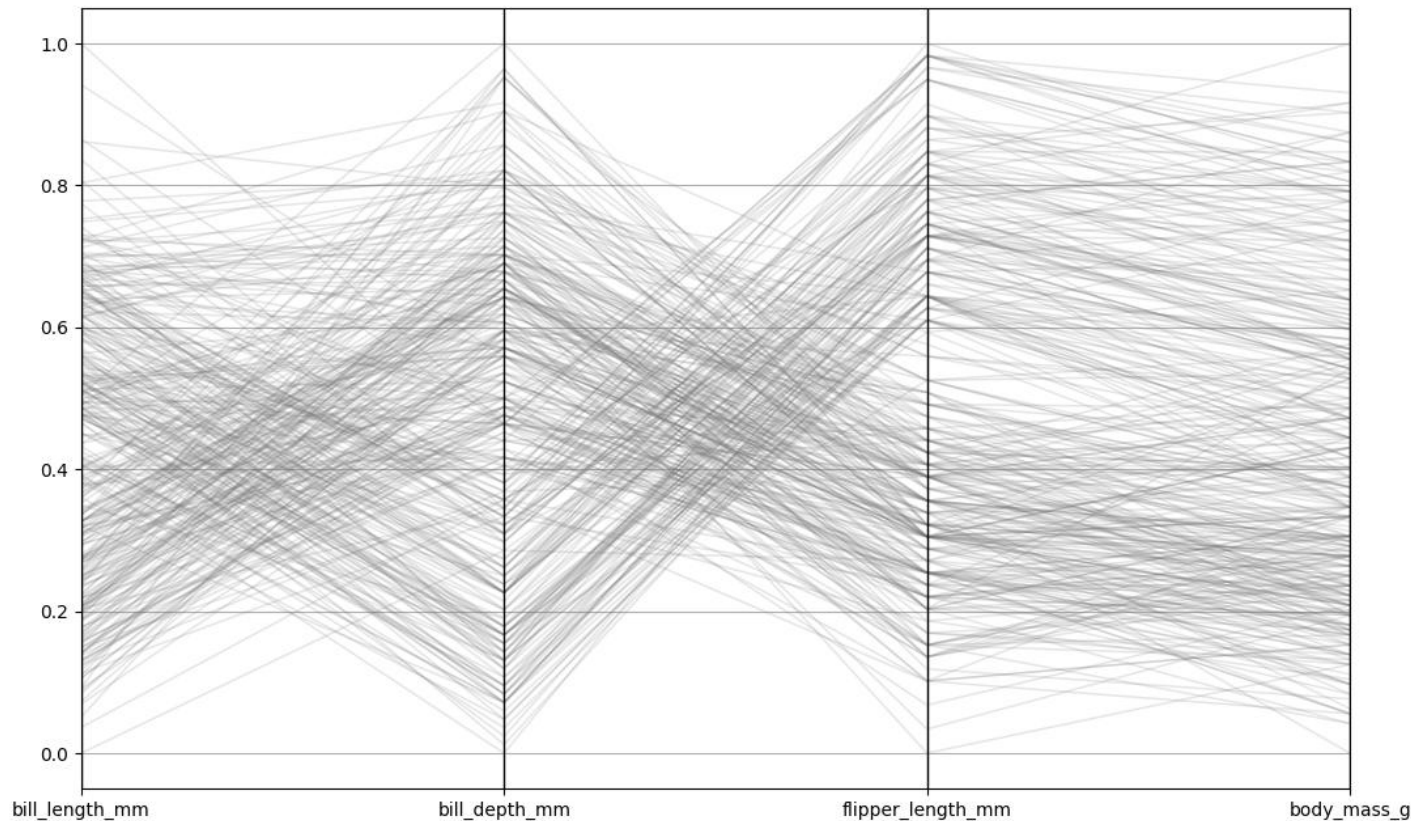
mínimo → 0

máximo → 1

- ¿Qué podemos descubrir si representamos todos los pingüinos?

Coordenadas Paralelas

- Cada línea representa ahora un pingüino



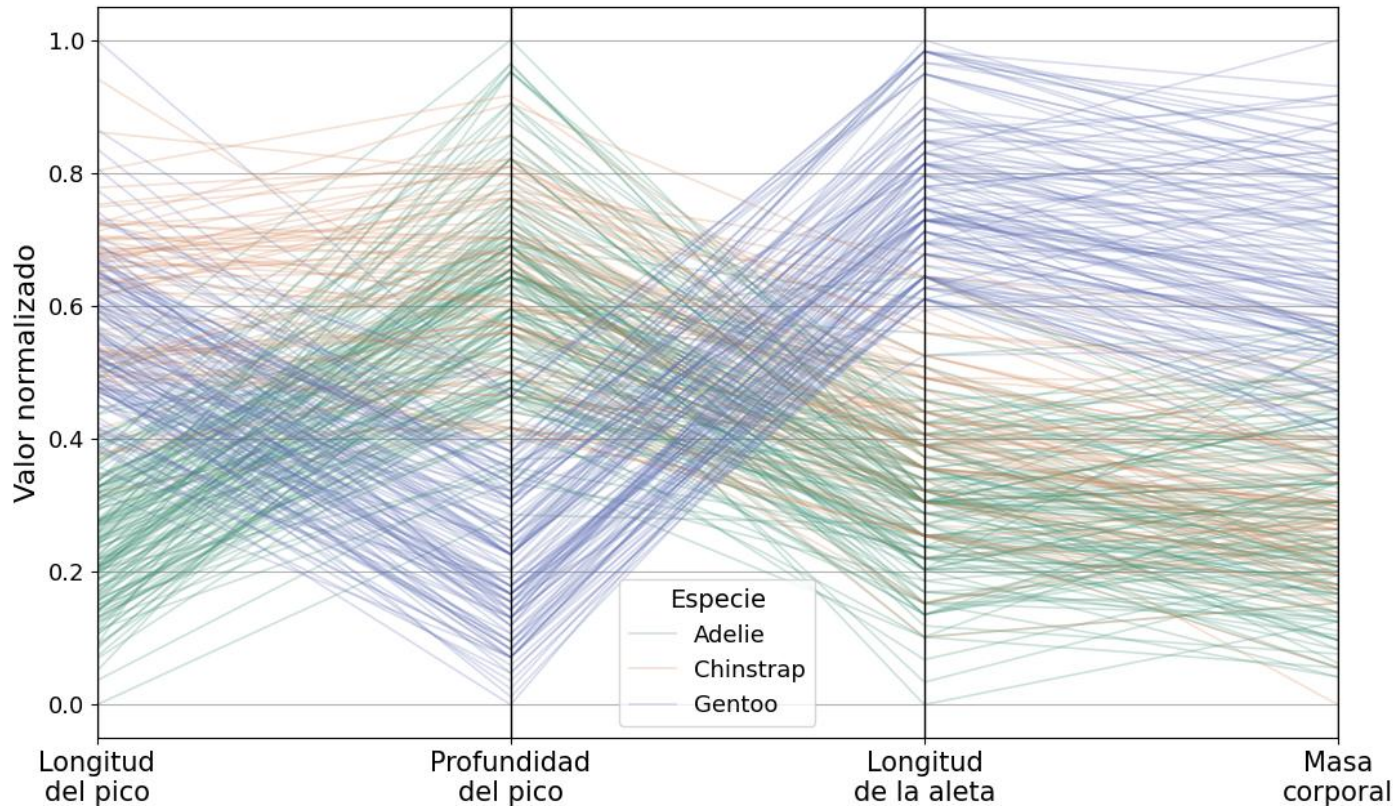
¿Qué podemos descubrir al representar muchas observaciones simultáneamente?



- Con muchas observaciones aumenta el **solapamiento** y resulta difícil seguir líneas individuales

Coordenadas Paralelas

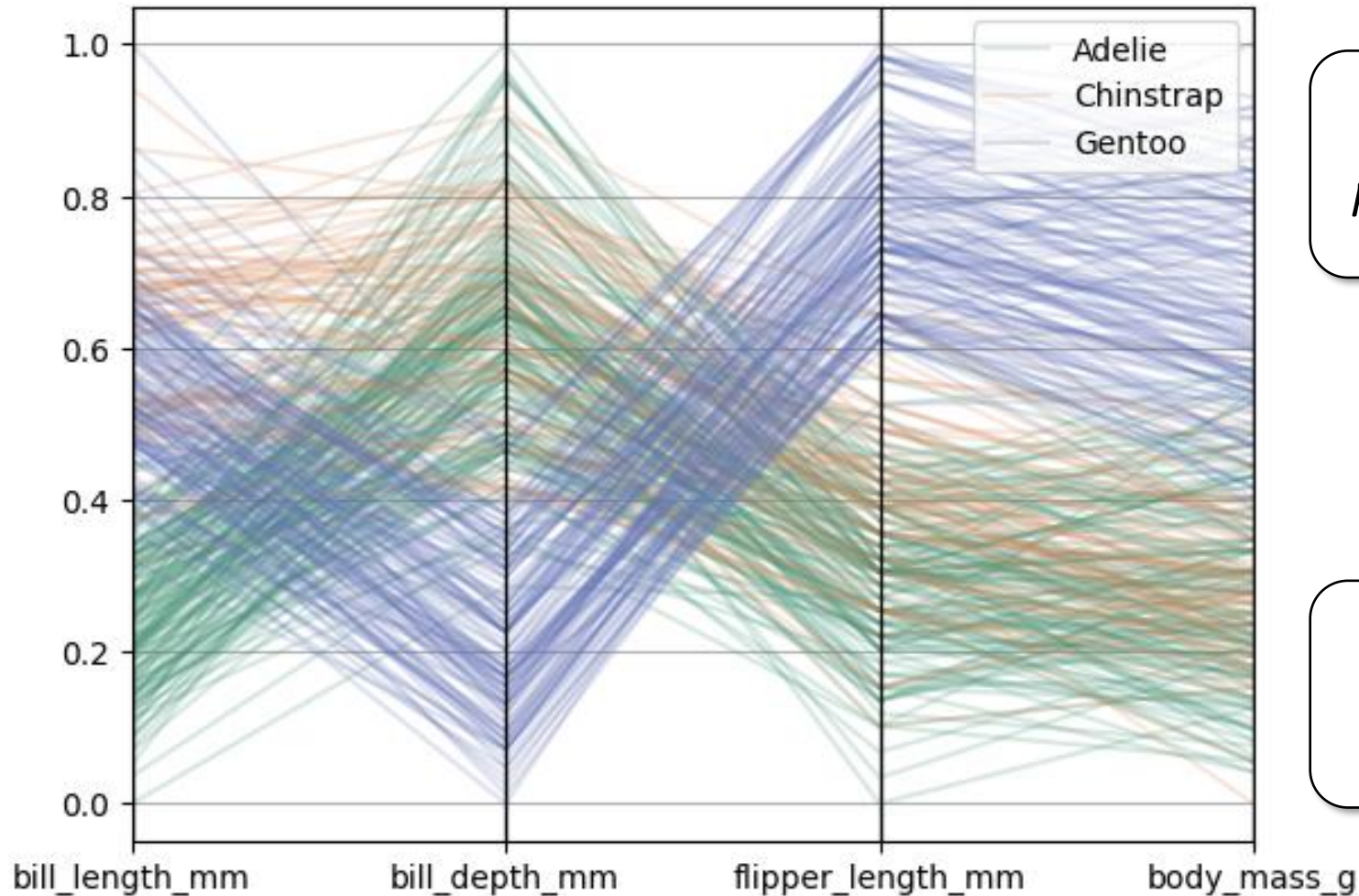
► Cada línea representa ahora un pingüino con mapeo Especie → color



*¿Aparecen perfiles diferentes según la especie?
Gentoo: mayor longitud de aleta y masa corporal*

- **¿Qué podemos observar?**
- ✓ **Niveles:** valores altos o bajos en cada atributo
 - ✓ **Perfiles:** recorridos similares a través de varios atributos
 - ✓ **Grupos:** conjuntos de líneas con comportamientos semejantes
 - ✓ **Solapamientos:** regiones donde los grupos resultan difíciles de distinguir

Coordenadas Paralelas – Orden de Ejes



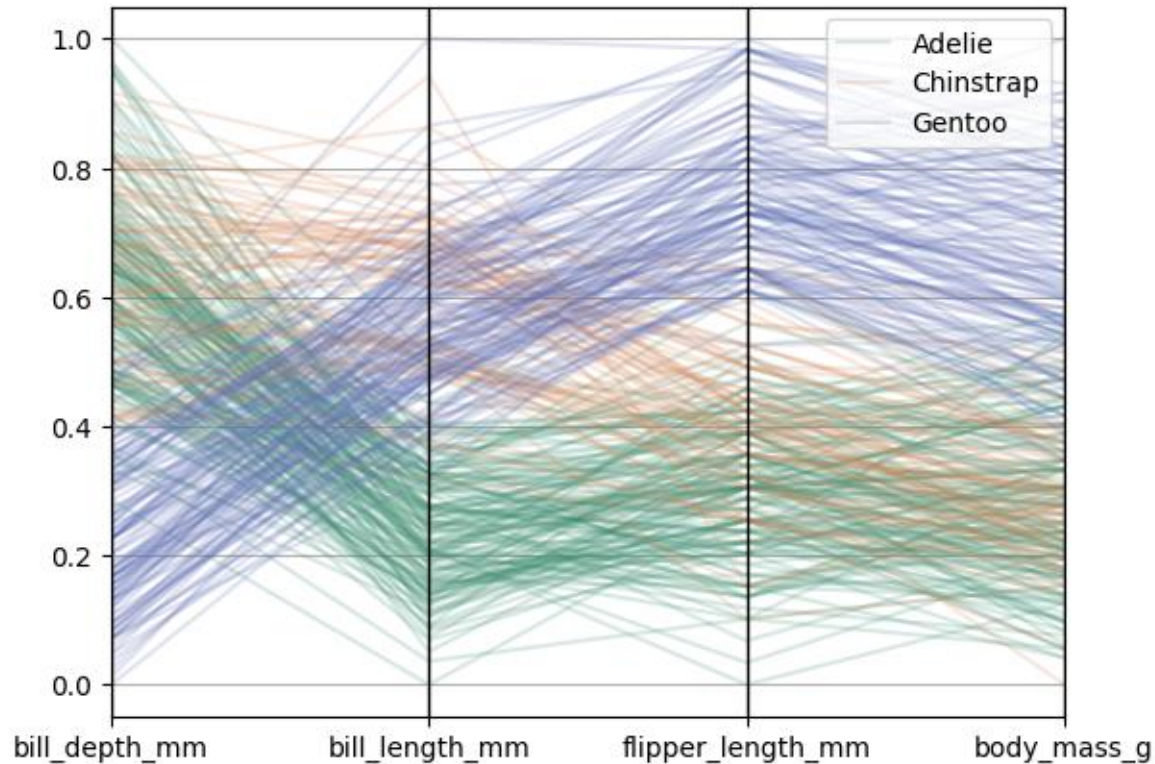
Las relaciones se perciben principalmente entre ejes adyacentes



El orden de los ejes condiciona qué relaciones resultan más visibles

Coordenadas Paralelas – Orden de Ejes

► Reordenamiento de los atributos



✓ Ayuda

Ordenar los ejes según la tarea: colocar próximas las dimensiones que interesa comparar

Usar transparencia: reduce el efecto del solapamiento

Usar color para grupos relevantes: facilita comparar perfiles

Normalizar las escalas: facilita comparar posiciones relativas

✗ Dificulta

Demasiadas observaciones: aumenta el solapamiento

Demasiadas dimensiones: aumenta la complejidad visual

Demasiados colores o estilos: dificulta seguir los patrones

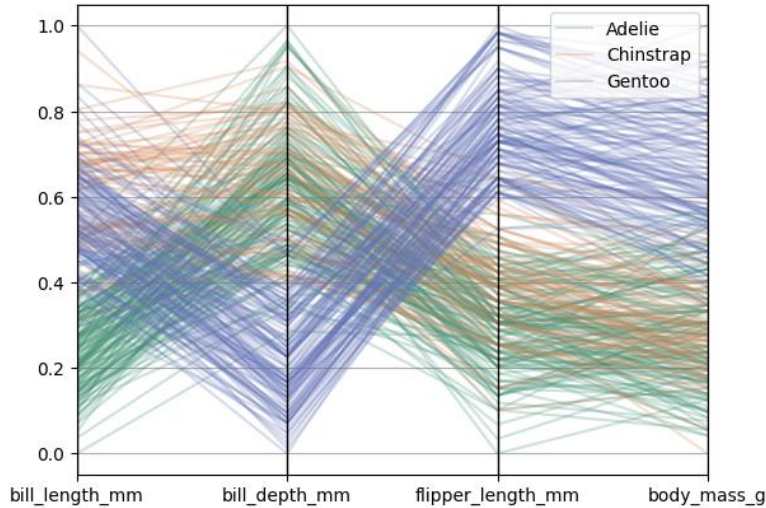
Orden arbitrario de ejes: puede ocultar relaciones relevantes

Aleta ↔ Masa corporal: $r \approx 0,87$

Al ubicarlas en ejes adyacentes, sus valores relativos similares generan segmentos con **pocos cruces**

- Entre ejes adyacentes, pocos cruces sugieren una relación positiva; cruces sistemáticos pueden indicar una relación negativa

Coordenadas Paralelas - Herramientas

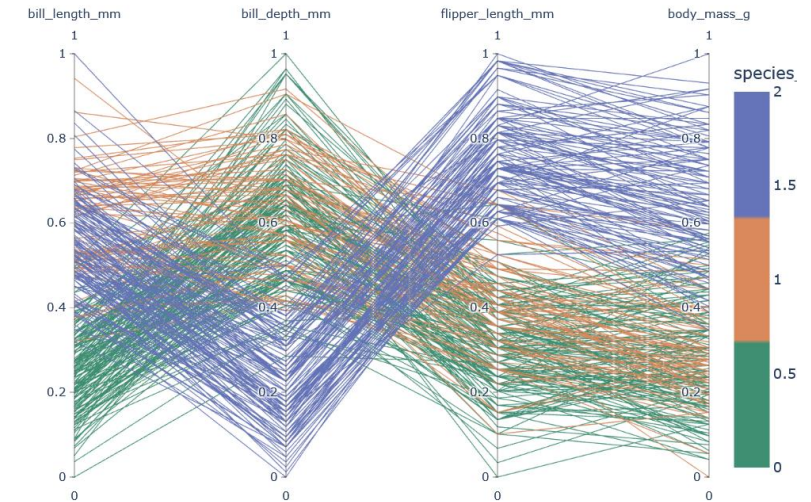


Matplotlib (Pandas)

```
import pandas.plotting
pandas.plotting.parallel_coordinates(
    data,
    "species",
    cols=variables,
    color=palette,
    alpha=0.25
)
```

Seaborn

seaborn no implementa
coordenadas paralelas

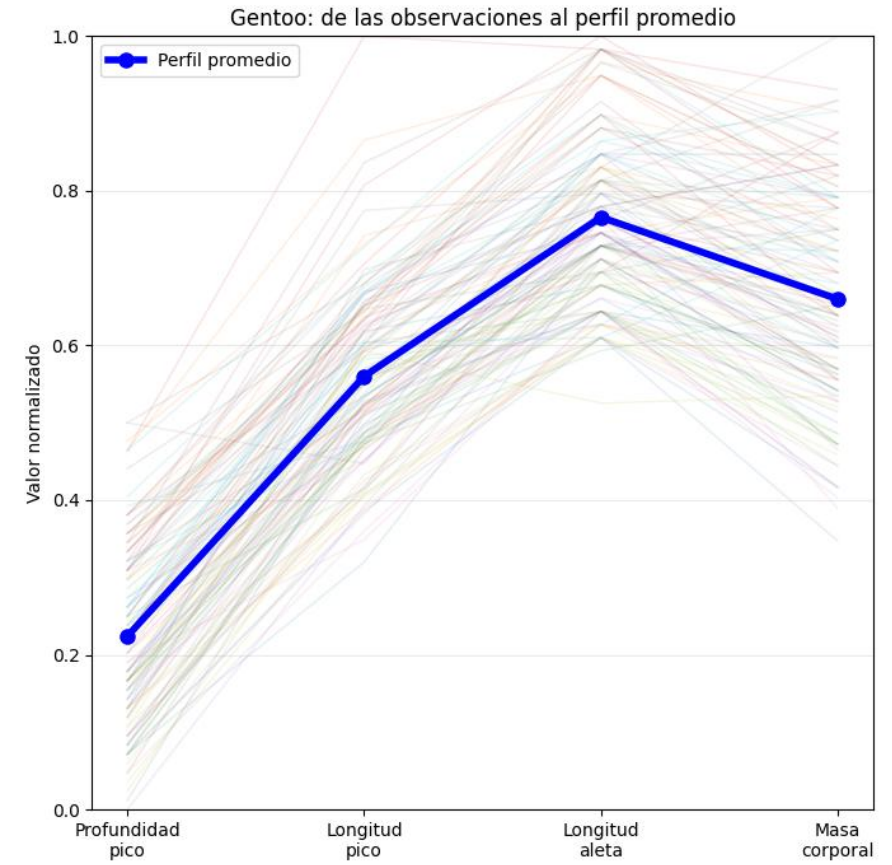


Plotly

```
# requiere valor continuo en data
# para colorear
fig = px.parallel_coordinates(
    data,
    dimensions=variables,
    color="species_code",
    color_continuous_scale=colors,
    range_color=[0, 2],
)
```

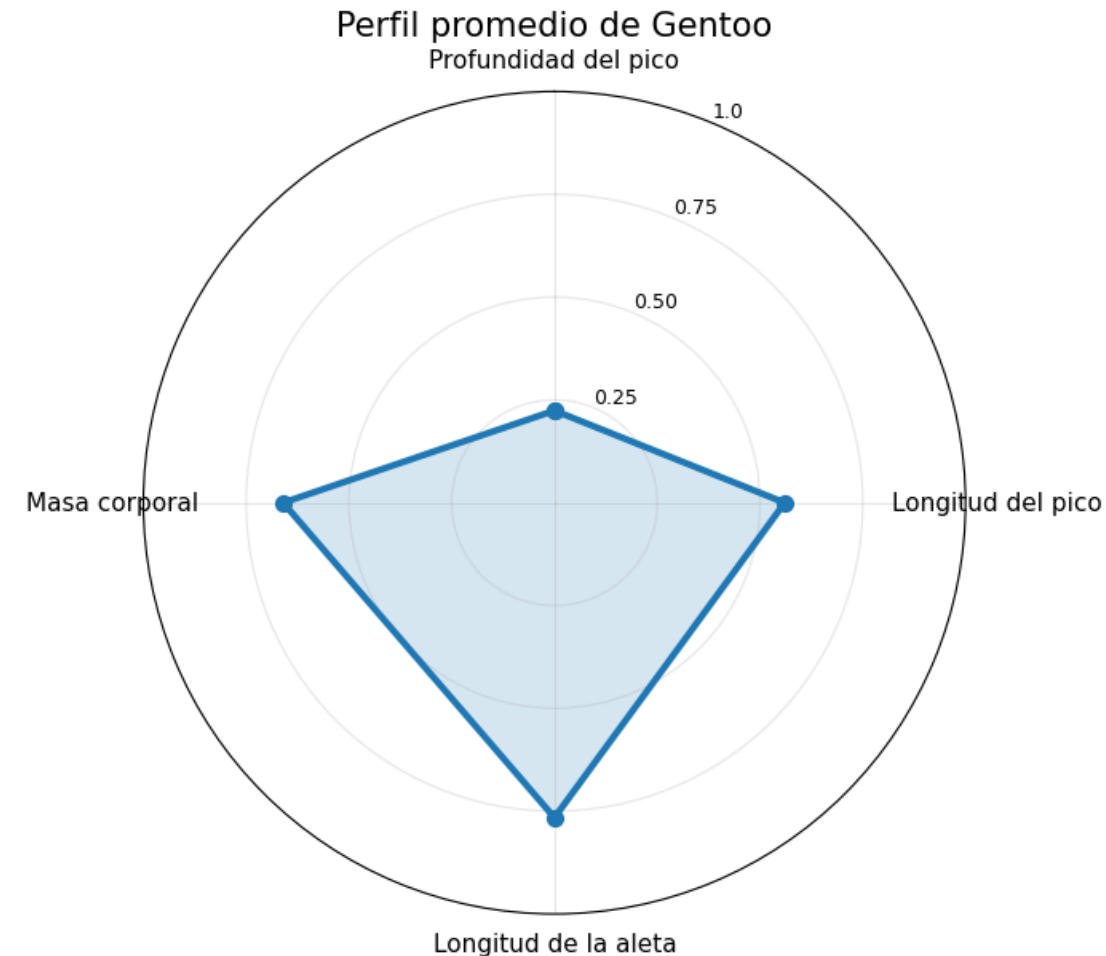
Resumiendo un Perfil Multidimensional

- ▶ En coordenadas paralelas, cada línea representa una observación mediante sus valores en varios atributos
- ▶ Cuando nos interesa caracterizar un **grupo**, podemos resumir sus observaciones:
 - ✓ una especie → múltiples pingüinos
 - ✓ múltiples valores por atributo → un valor representativo
 - ✓ varios atributos → un **perfil multidimensional**



Radar – Representar un Perfil Multidimensional

- ▶ Un gráfico de radar dispone varios **ejes alrededor de un centro común**
- ▶ Cada eje representa un atributo y la posición sobre él representa su valor
- ▶ Los valores de un mismo perfil se conectan formando una figura cerrada
- ▶ Para comparar atributos con distintas unidades y rangos necesitamos utilizar una **escala común**
- ▶ Perfil en coordenadas paralelas → mismos valores → ejes dispuestos radialmente: mismos datos, nueva disposición



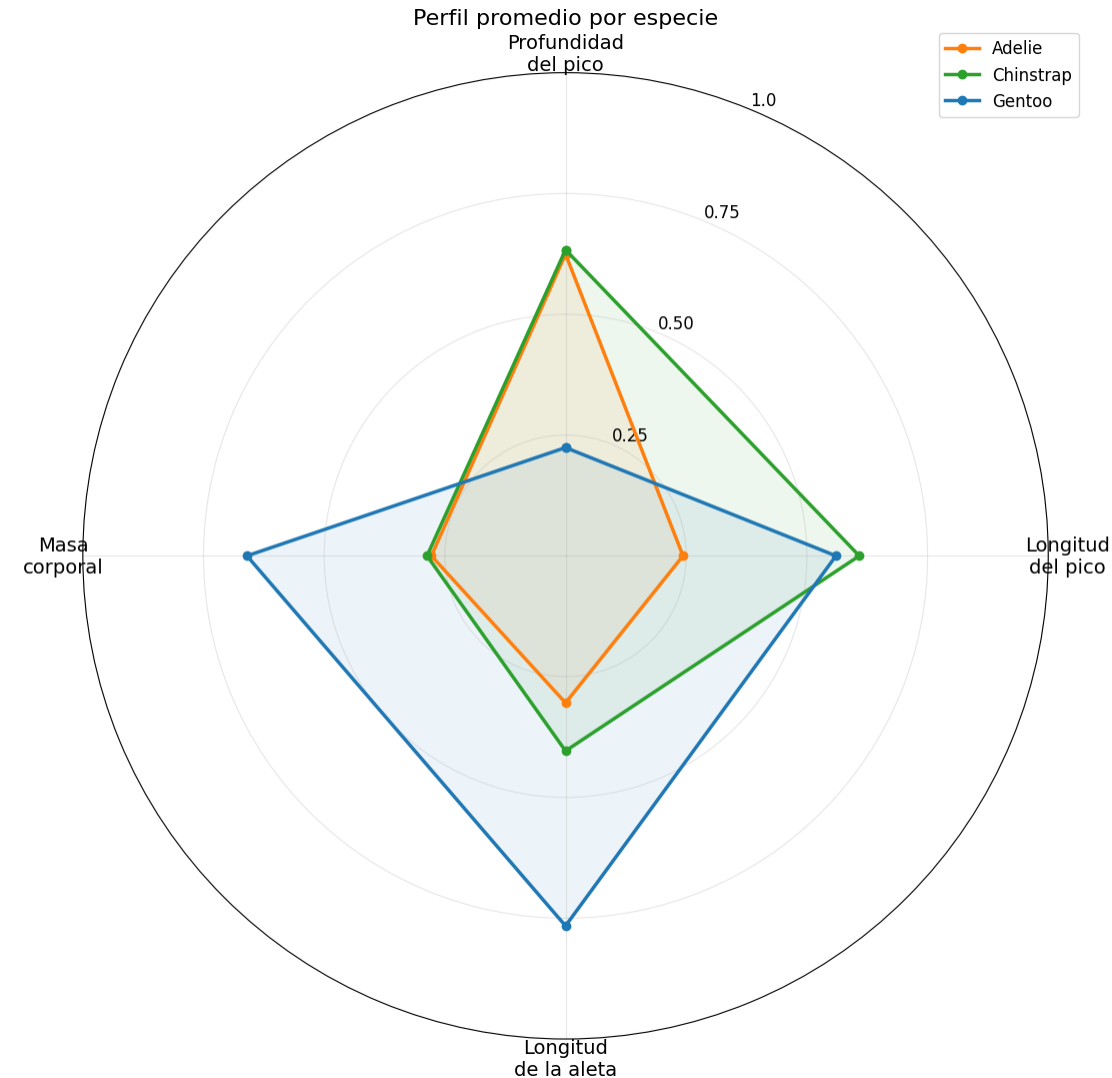
Radar – Comparar Perfiles

- Podemos construir un perfil representativo para cada grupo utilizando los mismos atributos y escalas que coordenadas paralelas
- Al representarlos sobre los mismos ejes podemos observar:
 - ✓ atributos en los que los grupos presentan valores similares
 - ✓ atributos que permiten diferenciarlos
 - ✓ patrones globales en sus perfiles



¿Qué atributos permiten diferenciar mejor a las especies?

El radar facilita comparar perfiles cuando el número de atributos y grupos es reducido



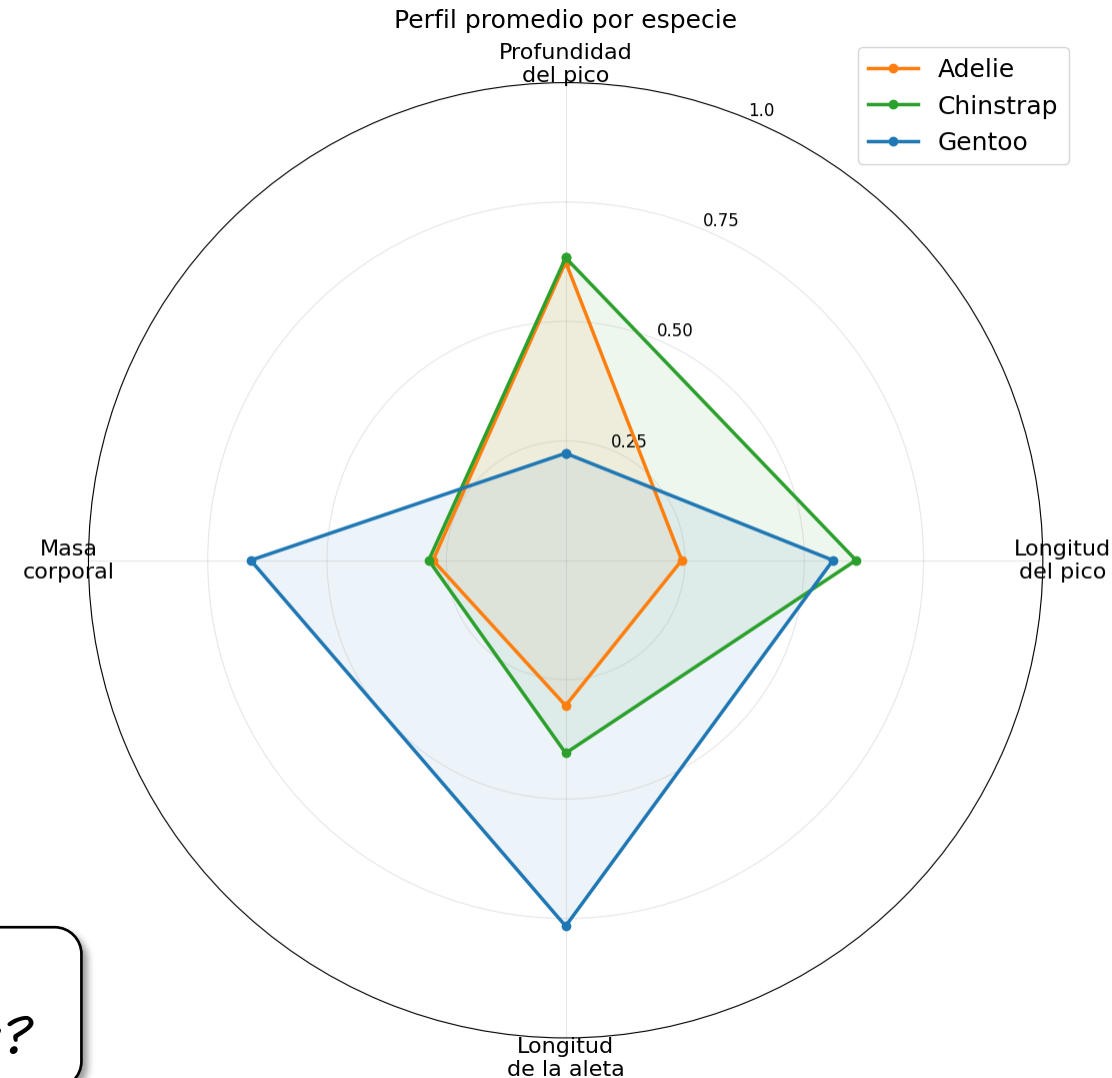
Radar – Interpretando Perfiles

► ¿Qué podemos observar?

- ✓ **Gentoo** se diferencia por mayores valores relativos de longitud de aleta y masa corporal
- ✓ **Chinstrap** presenta el mayor valor relativo de longitud del pico
- ✓ **Adelie** presenta menor longitud del pico, de la aleta y masa corporal
- ✓ Cada especie genera un **perfil característico** al considerar simultáneamente los cuatro atributos



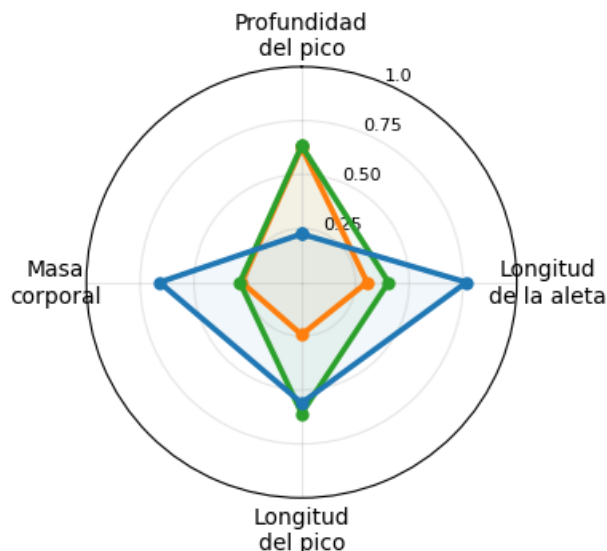
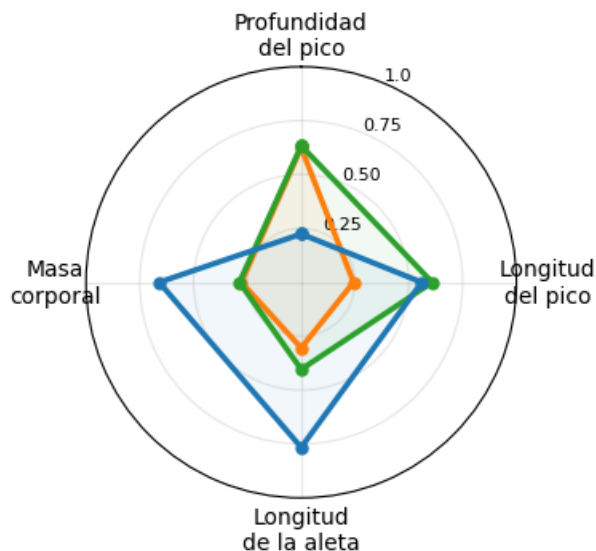
¿Qué sería más difícil de observar si agregáramos más atributos o más grupos?



Radar Alcances y Limitaciones

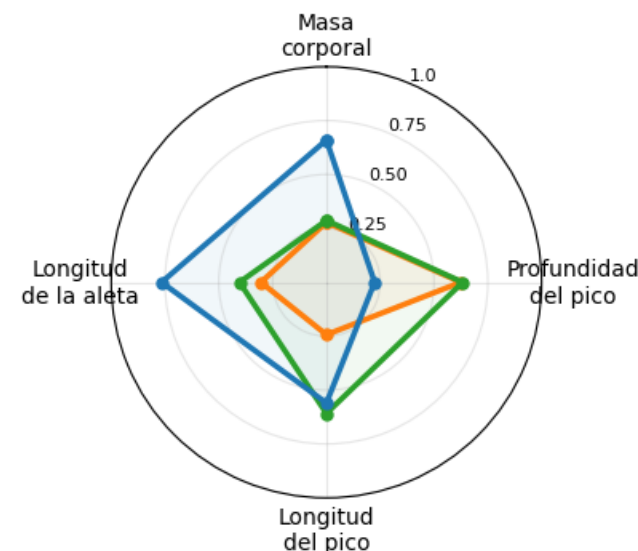
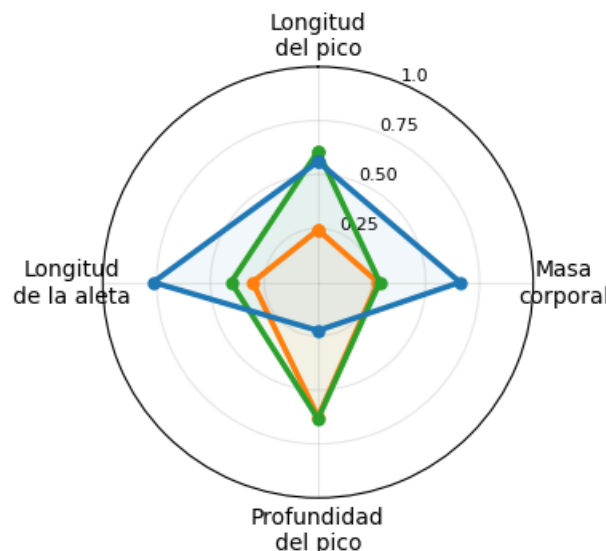
✓ Facilita

- ✓ Comparar **pocos perfiles** simultáneamente
- ✓ Reconocer **patrones globales**
- ✓ Identificar atributos que **caracterizan o diferencian** grupos
- ✓ Comunicar un **perfil multidimensional** de forma compacta



⚠ Dificulta

- ✓ Comparar **valores con precisión**
- ✓ Incorporar muchos atributos o perfiles sin generar **superposición**
- ✓ Interpretar correctamente los datos si las escalas no son comparables
- ✓ Mantener la misma percepción del perfil al cambiar el **orden de los ejes**



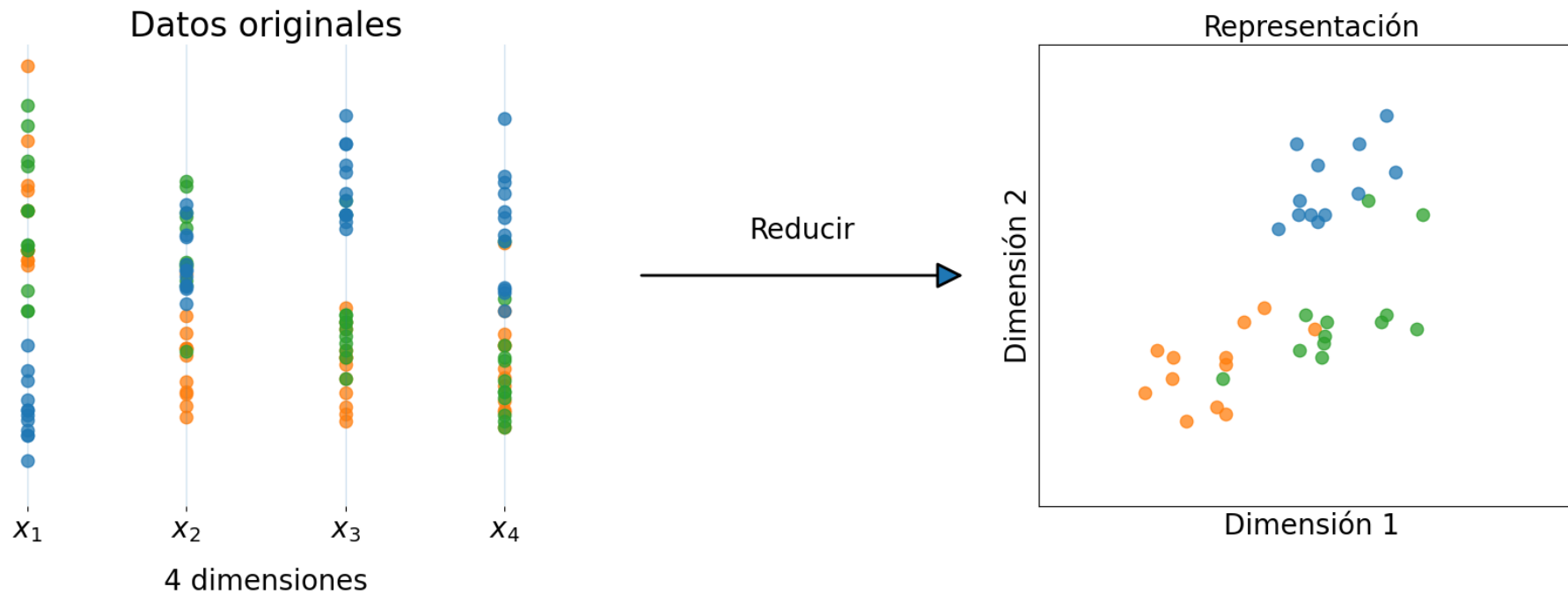
Reducir para Visualizar

- ▶ Conforme aumenta el número de atributos, representar todas las dimensiones simultáneamente se vuelve más difícil
- ▶ Podemos:
 - ✓ **Representar las dimensiones originales** usando Coordenadas paralelas / Radar
↓ **o cambiar el problema** ↓
 - ✓ Construir una representación con menos dimensiones
- ▶ La **reducción de dimensionalidad** transforma los datos para obtener una representación de menor dimensión que conserve determinadas propiedades de los datos



Reducir para Visualizar

- ▶ Reducir dimensiones implica decidir qué propiedades intentamos preservar
- ▶ Un total de N dimensiones se proyecta a 2D/3D
- ▶ Existen varias técnicas de reducción de dimensionalidad



Proyección de los Datos

- ▶ Cada observación está definida originalmente por múltiples atributos
- ▶ Una proyección asigna a cada observación una nueva posición en un espacio de menor dimensión (por ejemplo **2D**)

$$(x_1, x_2, \dots, x_n) \rightarrow \text{Proyección} \rightarrow (z_1, z_2)$$

- ▶ Al reducir dimensiones, parte de la información original se pierde
- ▶ No existe una única forma de reducir dimensiones: distintos métodos intentan preservar distintas propiedades

Si tenemos que perder información, ¿qué deberíamos intentar conservar?

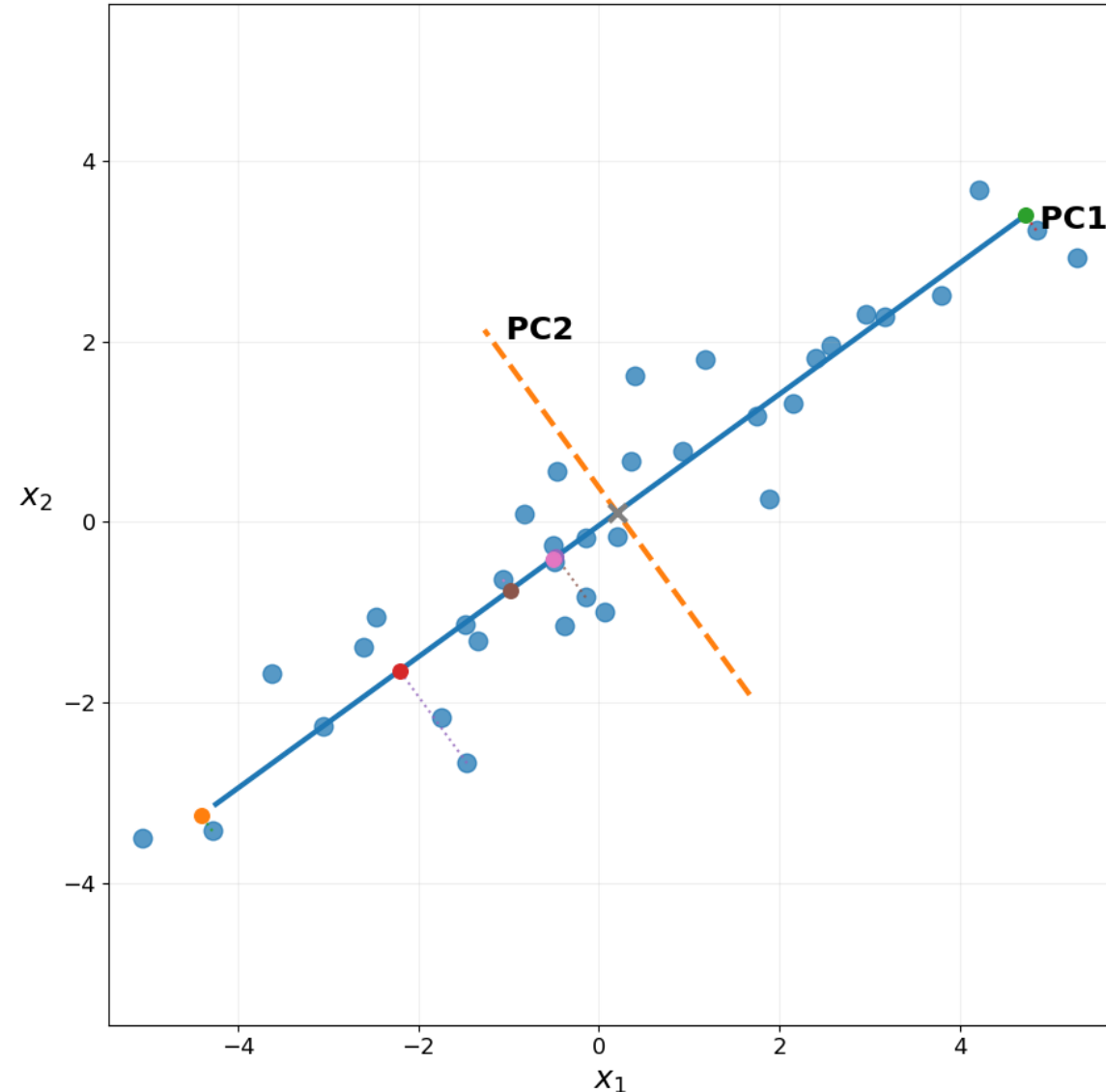


- ✓ diferencias entre observaciones
- ✓ relaciones entre observaciones
- ✓ estructura o patrones presentes en los datos

PCA - Buscando la Mayor Variabilidad

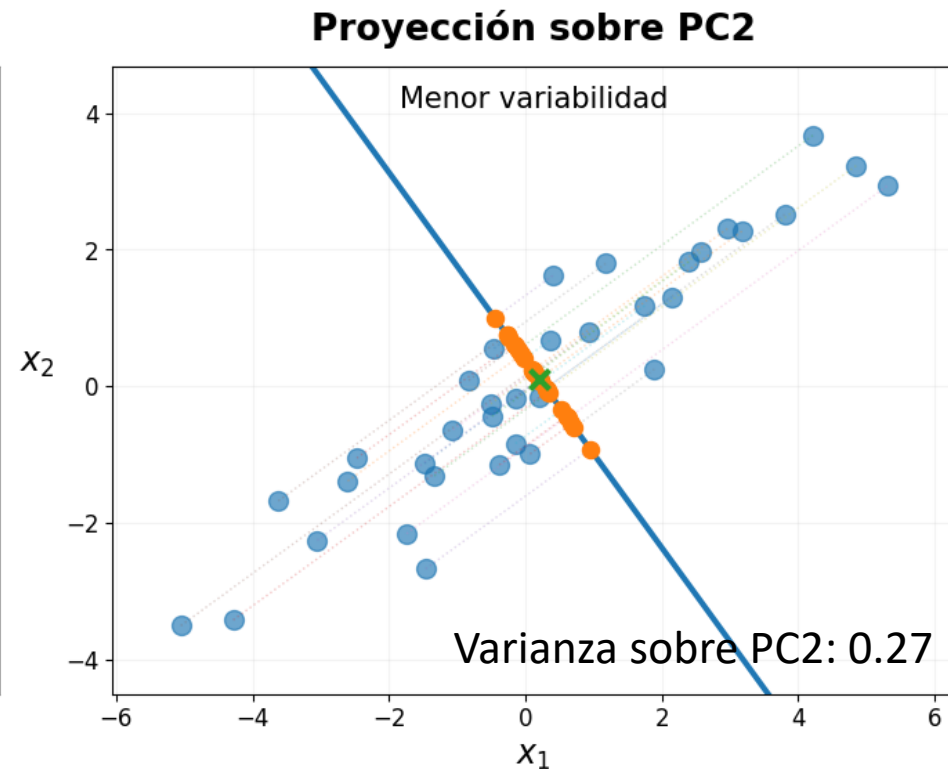
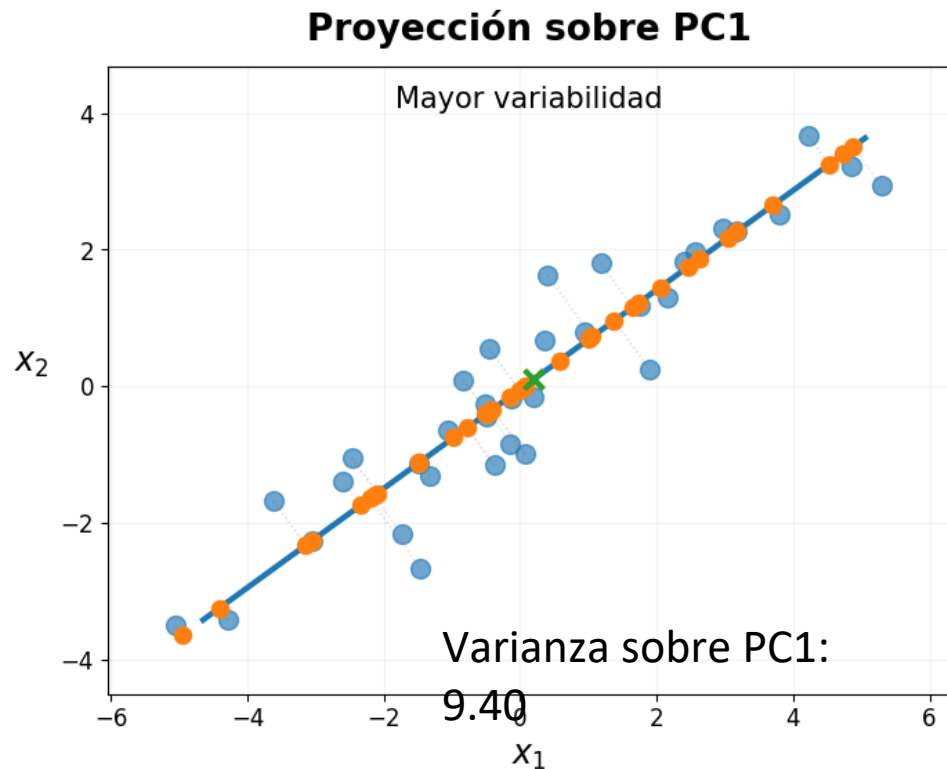
- ▶ Análisis de Componentes Principales (PCA) busca nuevas direcciones que permitan describir la **variabilidad** de los datos
- ▶ La primera componente principal (**PC1**) es la dirección en la que los datos presentan mayor variabilidad
- ▶ La segunda (**PC2**) es perpendicular a la primera y captura la mayor variabilidad restante

$$(x_1, x_2, x_3, x_4) \rightarrow (PC_1, PC_2)$$



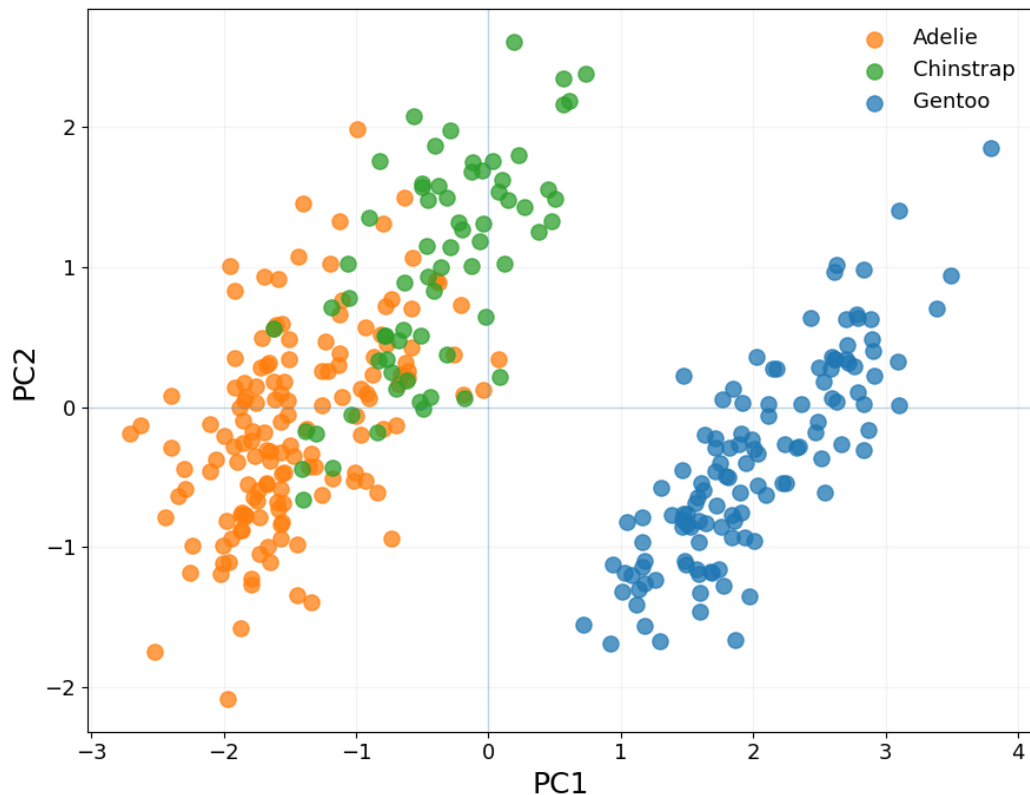
PCA - Conservando la Mayor Variabilidad

- Diferentes direcciones producen diferentes proyecciones de los mismos datos
- PCA elige como **PC1** la dirección que maximiza la variabilidad de los datos proyectados y PC2 captura la mayor variabilidad restante (perpendicular a PC1)



PCA - De Cuatro Atributos a Dos Componentes

- ▶ Cada pingüino, originalmente descrito mediante **4 valores**, pasa a tener una posición definida por **PC1 y PC2**
- ▶ Representamos esas nuevas coordenadas mediante un **gráfico de dispersión**



PCA utiliza los atributos cuantitativos; la especie se agrega después como color para interpretar la representación

PCA es sensible a si no se normaliza

Antes de construir el gráfico, los datos deben normalizarse. Es conveniente utilizar normalización estándar (media y desvío)

PCA - Varianza Explicada

- ▶ La **varianza explicada** indica qué proporción de esa variabilidad es capturada por cada componente
- ▶ Al conservar PC1 y PC2, nuestra representación bidimensional captura aproximadamente el **88.2 % de la variabilidad**:

PC1	PC2	PC3	PC4
68.8%	19.3%	9.1%	2.7%

- ▶ Reducir dimensiones implica un compromiso entre simplificar la representación y conservar la variabilidad de los datos

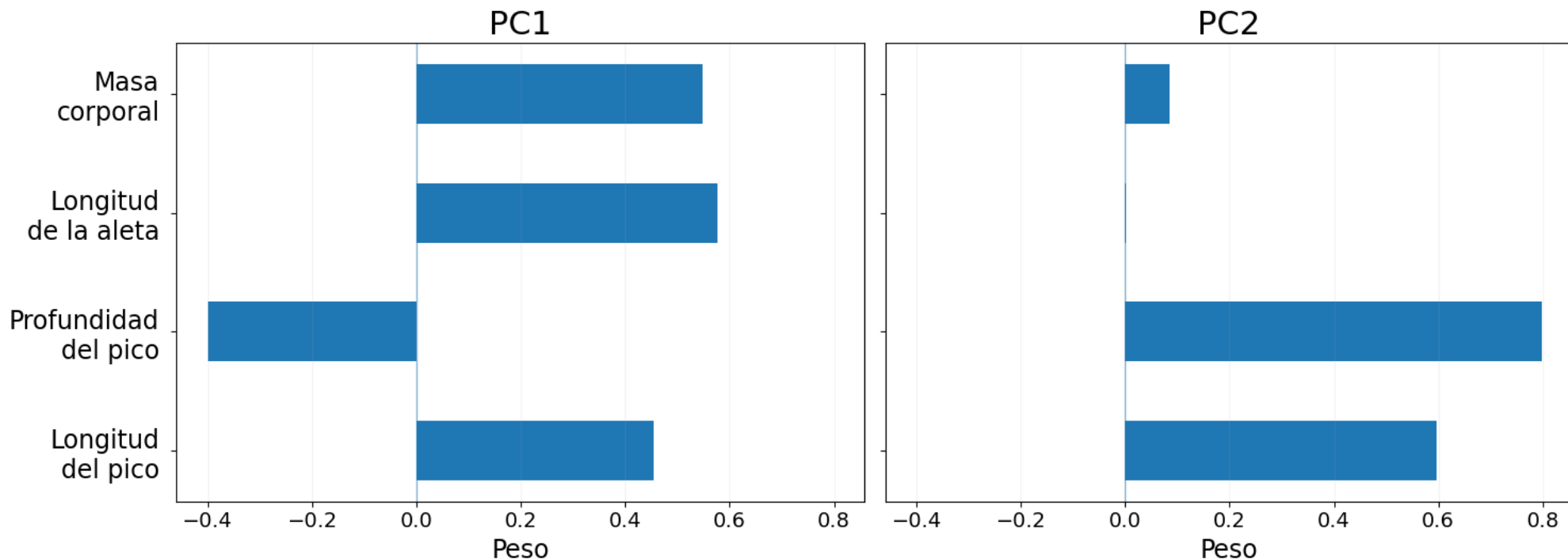
*¿Es suficiente
conservar el 88.2 %
de la variabilidad?*



PCA – Interpretando las Componentes

- ▶ Cada componente combina los atributos originales asignándoles distintos **pesos**
- ▶ Podemos expresarlo conceptualmente como:

$$PCi = a_{i1} \cdot \text{bill length} + a_{i2} \cdot \text{bill depth} + a_{i3} \cdot \text{flipper} + a_{i4} \cdot \text{body mass}$$



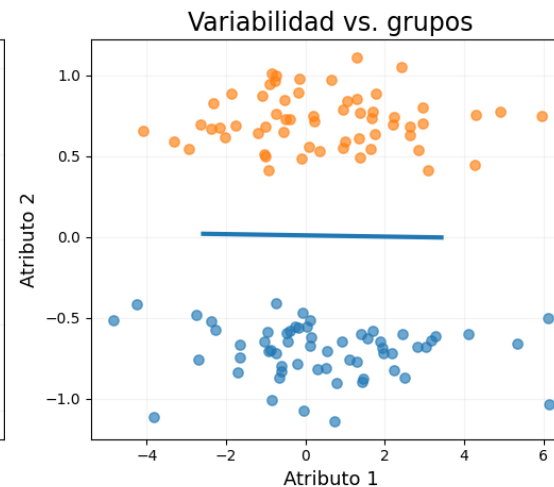
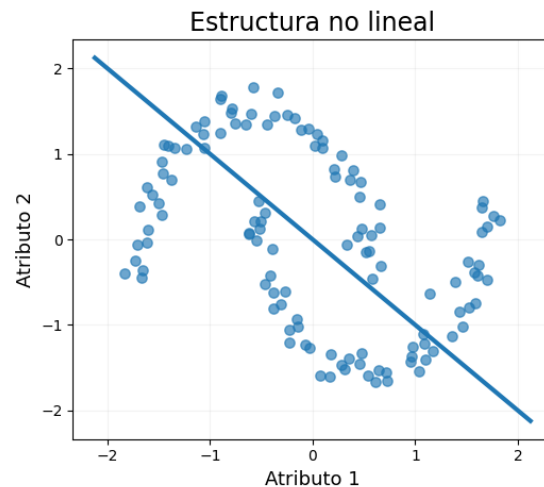
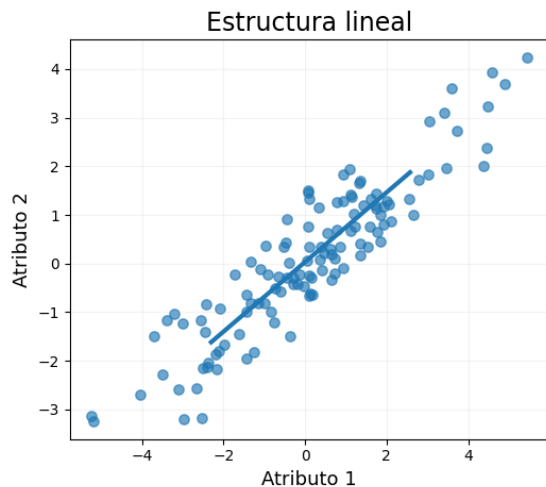
Longitud de aleta, masa corporal y longitud del pico contribuyen de forma importante

Dominada por profundidad y longitud del pico.
Longitud de aleta y masa corporal no contribuyen

PCA - Alcances y Limitaciones

✓ Facilita

- ✓ **Reducir** muchas dimensiones a unas pocas componentes
- ✓ **Concentrar variabilidad** en las primeras componentes
- ✓ **Visualizar estructuras** que no podemos observar directamente en muchas dimensiones
- ✓ Detectar **patrones, agrupamientos y tendencias** en la representación reducida



⚠ Limita

- ✓ Las componentes son **combinaciones de atributos** y pueden ser difíciles de interpretar
- ✓ Al reducir dimensiones se **pierde información**
- ✓ PCA captura principalmente **estructura lineal**
- ✓ Es **sensible a la escala** de los atributos → de ahí la estandarización previa
- ✓ Conservar mucha variabilidad **no garantiza separar grupos o categorías**

PCA encuentra direcciones lineales: puede ocultar estructuras no lineales presentes en los datos

PCA – Ejemplo Completo

Python

```
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

```
penguins = sns.load_dataset("penguins").dropna()
```

```
cols = ["bill_length_mm", "bill_depth_mm",
        "flipper_length_mm", "body_mass_g"]
```

```
X = StandardScaler().fit_transform(penguins[cols])
```

```
pca = PCA(n_components=2)
Z = pca.fit_transform(X)
```

```
penguins["PC1"] = Z[:, 0]
penguins["PC2"] = Z[:, 1]
```

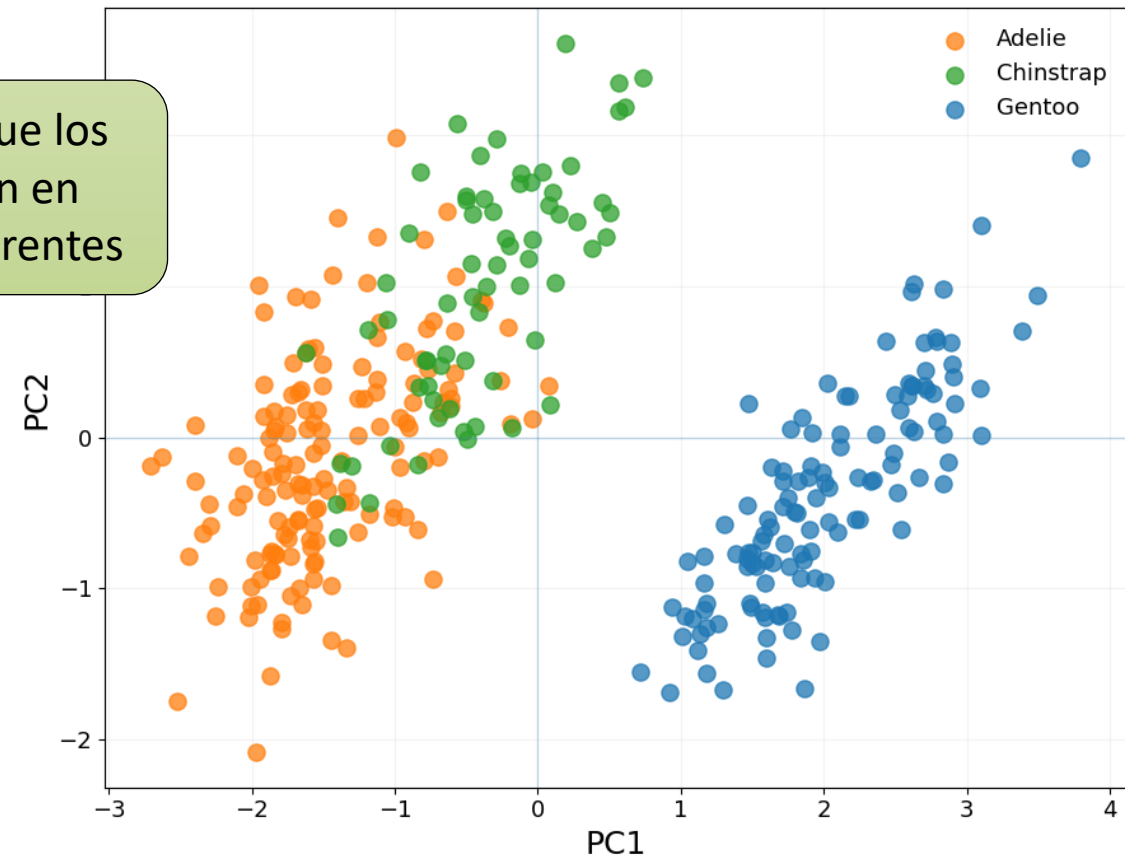
```
sns.scatterplot(data=penguins, x="PC1", y="PC2",
                hue="species")
```

```
print(pca.explained_variance_ratio_)
```

Necesario porque los atributos están en escalas muy diferentes

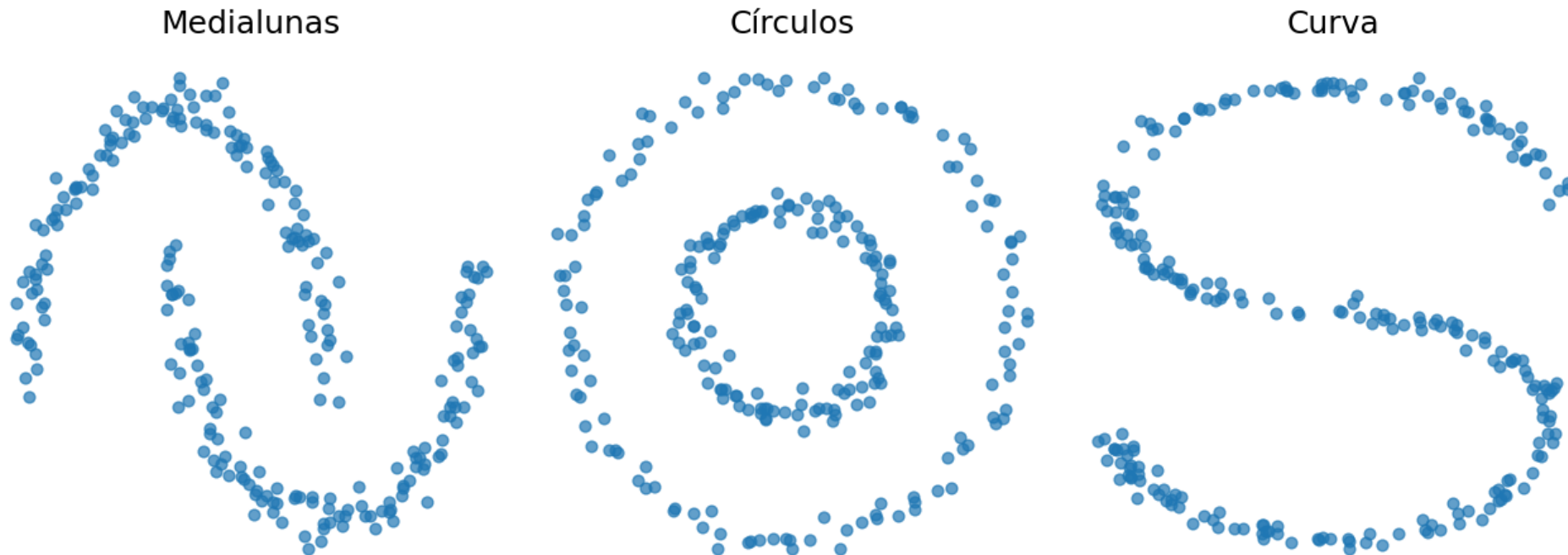
Calcula componentes principales 2D

PC1	PC2
[0.686338	0.194529]



Reducción No Lineal

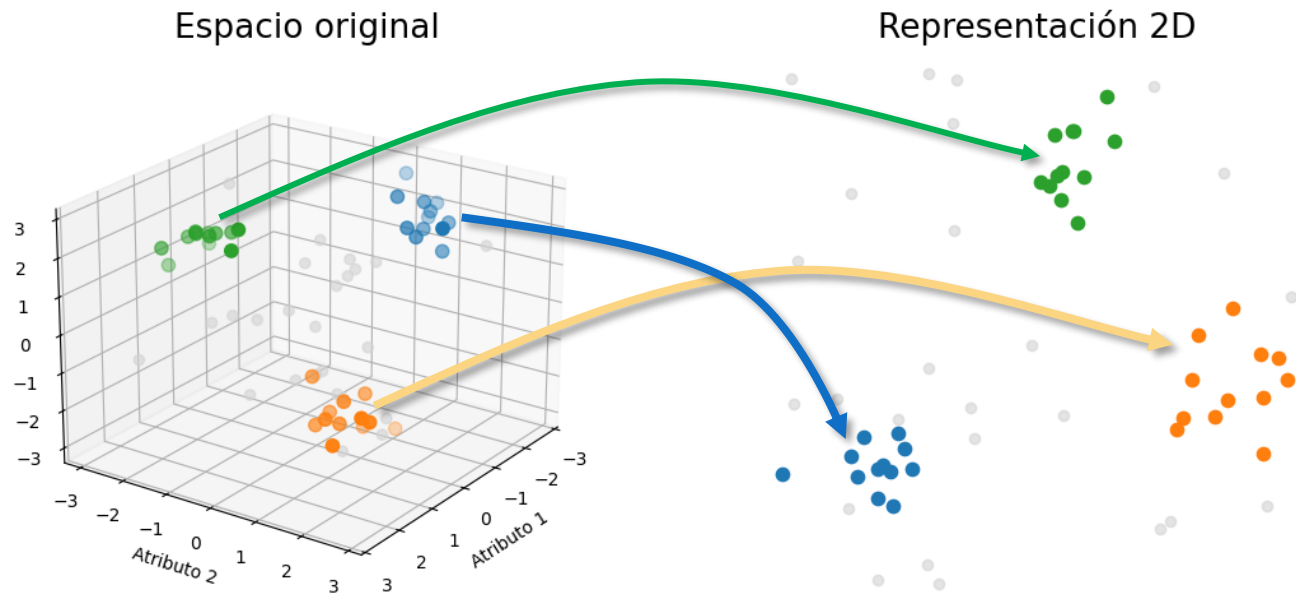
- ▶ **PCA** busca nuevas direcciones mediante **combinaciones lineales** de los atributos
- ▶ No toda estructura de los datos puede describirse mediante direcciones lineales



- ▶ Se requieren técnicas de reducción que representen en pocas dimensiones **relaciones más complejas entre las observaciones**

t-SNE - Preservar Vecindades

- ▶ En lugar de buscar direcciones de máxima variabilidad, t-SNE intenta que observaciones **similares permanezcan próximas** en la representación reducida
- ▶ Se concentra especialmente en preservar **relaciones locales**: quién está cerca de quién

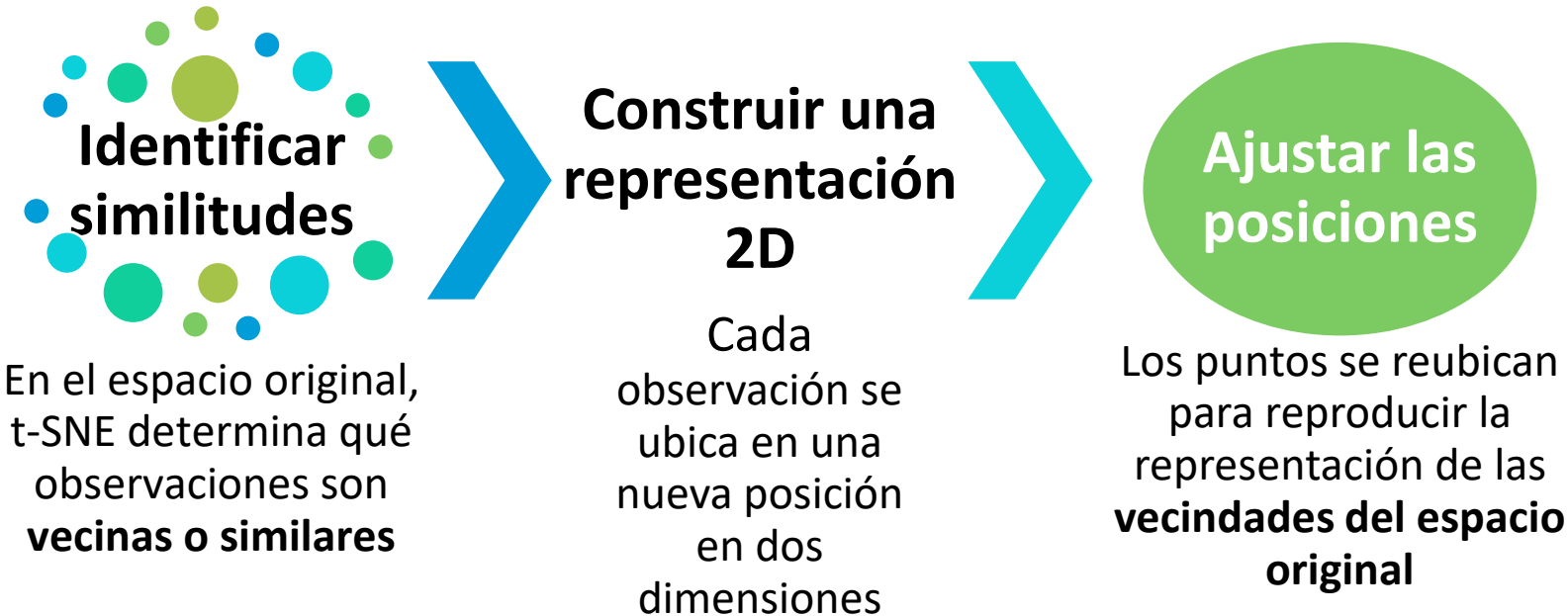


⚠ En el esquema los colores representan Vecindad (NO categorías)

- ▶ Aunque la disposición global cambia al pasar a 2D, las observaciones que eran vecinas en el espacio original tienden a permanecer próximas

t-SNE - Preservar Vecindades

- T-SNE realiza tiene 3 etapas para generar la representación 2D:



¿Qué busca conservar t-SNE al reorganizar los puntos?

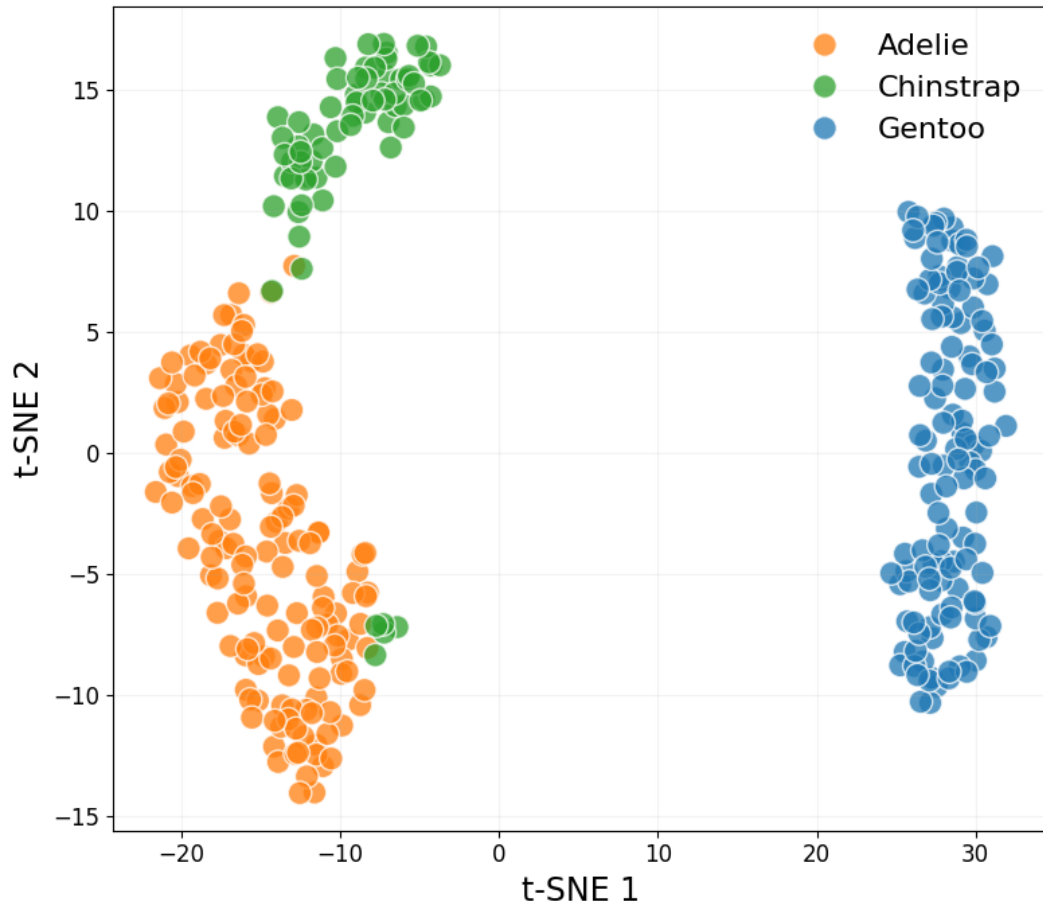


t-SNE optimiza la disposición de puntos para representar sus similitudes locales

- **Perplexity:** parámetro que controla la escala de vecindad:
 - ✓ **baja** → entorno más local
 - ✓ **alta** → entorno más amplio

t-SNE - De 4 atributos a 2 dimensiones

► 4 atributos → Estandarización → t-SNE → 2 dimensiones



► Scatterplot de t-SNE:

- ✓ cada punto → un pingüino
- ✓ X e Y → coordenadas generadas por t-SNE
- ✓ color → species



¿Qué estructura observamos ahora? ¿Cómo se compara con la representación obtenida mediante PCA?

PCA vs t-SNE - ¿Qué Intentamos Preservar?

PCA

Busca conservar variabilidad

- ✓ Transformación **lineal**
- ✓ Construye dimensiones: **componentes principales**
- ✓ PC1 y PC2 tienen una interpretación a partir de los atributos originales
- ✓ Permite cuantificar la **varianza explicada**
En PCA con penguins: **PC1 + PC2 = 88,2 %**

T-SNE

Busca conservar vecindades

- ✓ Transformación **no lineal**
- ✓ Optimiza la disposición de las observaciones en 2D
- ✓ Los ejes **no tienen una interpretación directa** porque los datos se posicionan según la vecindad
- ✓ La lectura se **centra** en la **estructura local**
- ✓ Las distancias y posiciones globales deben interpretarse con cautela

Mismos
Datos

Distinto
Objetivo

Distinta
Representación

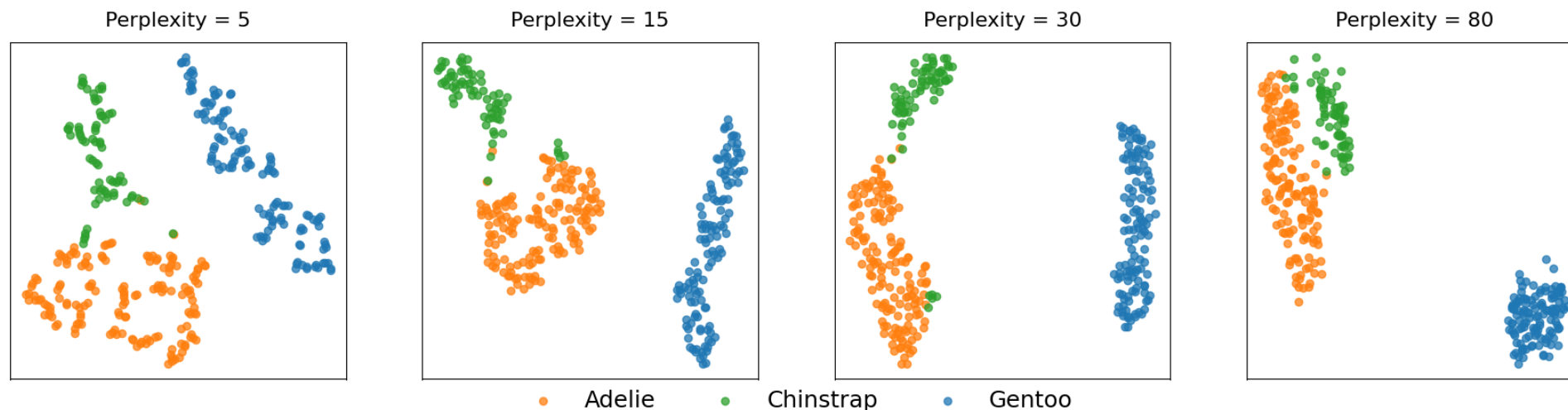
t-SNE - Alcances y Limitaciones

✓ Facilita

- ✓ Explorar **vecindades** entre observaciones
- ✓ Representar **estructuras no lineales**
- ✓ Detectar agrupamientos y patrones locales
- ✓ Visualizar datos con muchas dimensiones en 2D

⚠ Limita

- ✓ Los ejes **no tienen interpretación directa**
- ✓ Las **distancias globales** no deben interpretarse literalmente
- ✓ El resultado depende de parámetros como **perplexity**
- ✓ Cada ejecución puede producir disposiciones diferentes
- ✓ No genera algo similar a la **varianza explicada** de PCA



T-SNE - Ejemplo Completo

Python

```
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.manifold import TSNE

penguins = sns.load_dataset("penguins").dropna()

cols = ["bill_length_mm", "bill_depth_mm",
        "flipper_length_mm", "body_mass_g"]

X = StandardScaler().fit_transform(penguins[cols])

tsne = TSNE(n_components=2, perplexity=30,
            random_state=42)

Z = tsne.fit_transform(X)

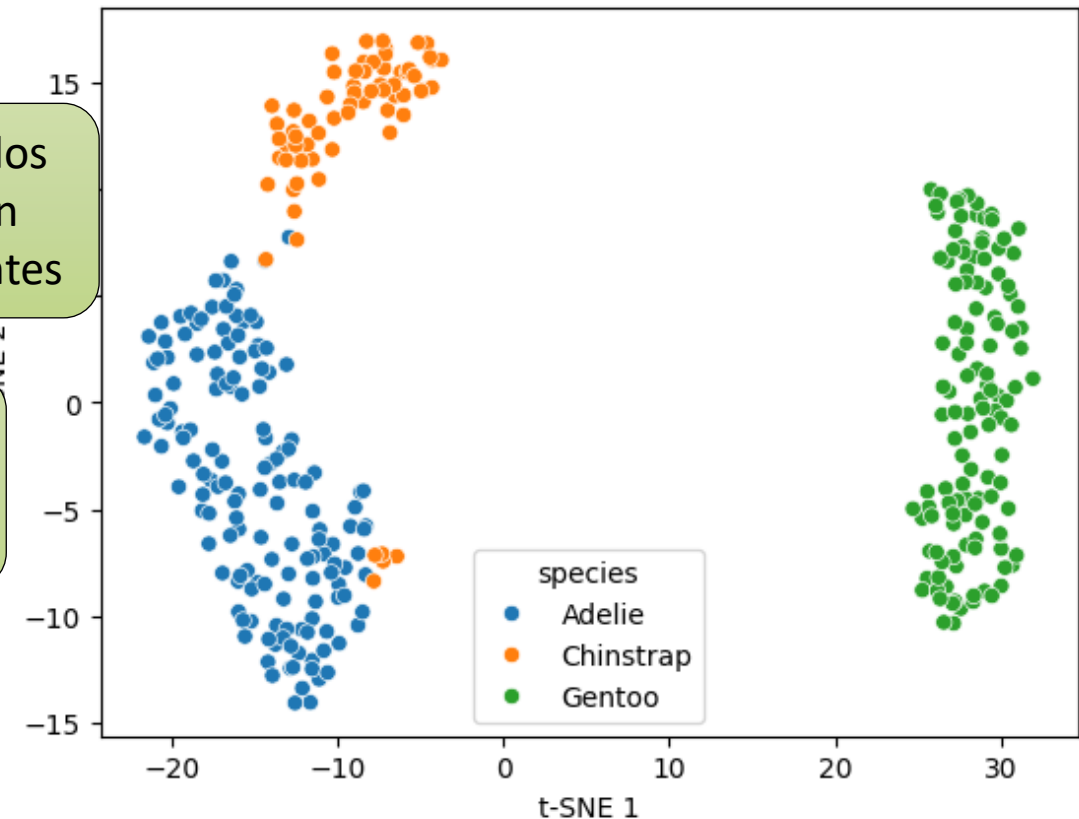
penguins["t-SNE 1"] = Z[:, 0]
penguins["t-SNE 2"] = Z[:, 1]

sns.scatterplot(data=penguins, x="t-SNE 1", y="t-SNE 2",
                hue="species")
```

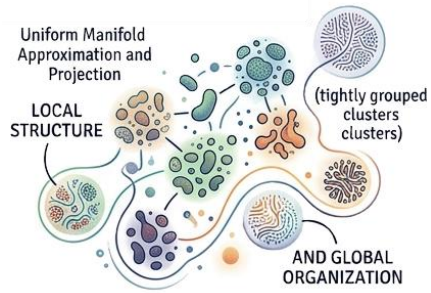
Necesario porque los atributos están en escalas muy diferentes

Configura para 2 componentes con "vecindad" de 30

Genera la representación 2D

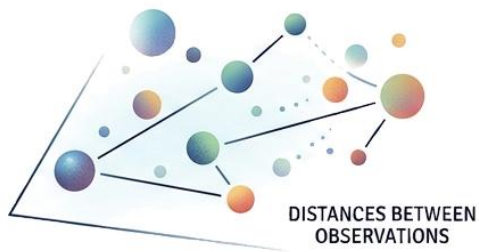


Reducción Dimensional - Otras Alternativas



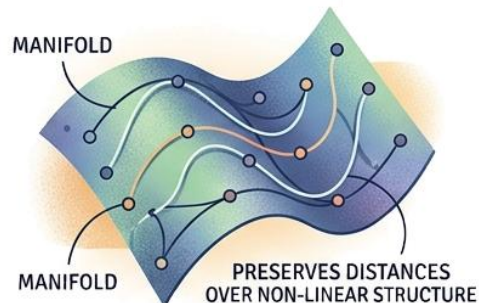
UMAP - Uniform Manifold Approximation and Projection

Construye relaciones de **vecindad** entre las **observaciones** y **busca reproducirlas en el espacio reducido**. Al igual que t-SNE, prioriza la estructura local, aunque intenta conservar mejor la organización a mayor escala. Suele ser computacionalmente más eficiente que t-SNE en datasets grandes.



MDS - Multidimensional Scaling

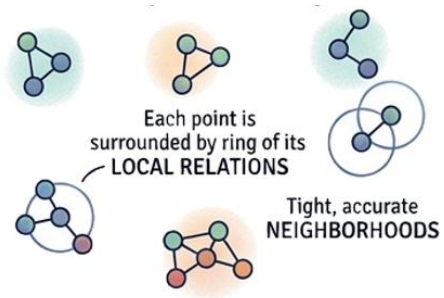
Su punto de partida son las **distancias** entre pares de **observaciones**. **Busca posiciones en un espacio** de menor dimensión de modo que las **distancias** allí sean **parecidas** posible a las **originales**. Es muy intuitivo y puede trabajar a partir de una matriz de distancias/disimilitudes, sin necesitar los atributos originales.



Isomap - Isometric Mapping

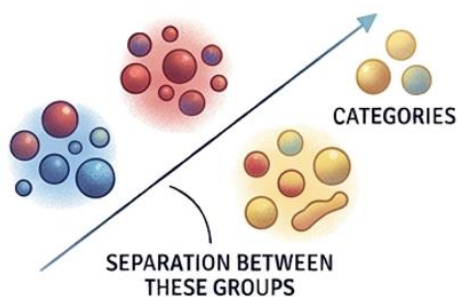
En lugar de utilizar la distancia euclídea directa entre dos puntos, **construye una red de vecinos** y **estima distancias** siguiendo **camino**s sobre esa estructura. En una superficie enrollada, dos puntos pueden estar espacialmente cerca atravesando la superficie, pero lejos al recorrerla superficie.

Reducción Dimensional - Otras Alternativas



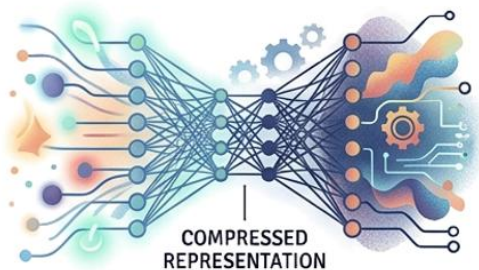
LLE - Locally Linear Embedding

Para cada observación busca vecinos y expresa ese punto como una **combinación de sus vecinos**. Después busca una **representación de menor dimensión** que **preserve esas relaciones locales**. No intento conservar todas las distancias, sino la relación local de cada punto con sus vecinos. Es similar a UMAP o t-SNE pero menos popular.



LDA — Linear Discriminant Analysis

Técnicamente LDA es un método de **clasificación supervisado** pero puede utilizarse para reducción dimensional. Busca una proyección de menor dimensión donde las categorías queden discriminadas. **Intenta favorecer una alta separación entre grupos y una baja dispersión dentro de cada grupo.**



Autoencoders

Son **redes neuronales** entrenadas para **reproducir su propia entrada**. Tienen dos partes: un *encoder* **comprime los datos** a una representación de menor dimensión (espacio latente) y un *decoder* intenta reconstruir los datos originales a partir de ella. Puede **aprender transformaciones no lineales** complejas. Necesita entrenamiento, más datos y decisiones de arquitectura, y puede ser **difícil de interpretar**.